# TRUSTCHAIN

**Final Project Report Submitted to**

**The Department of Computer Science**

**Faculty of Computer and Information Technology**

**Jordan University of Science and Technology**

**In Partial Fulfillment of the Requirements for the Degree of Bachelors of Science in**

**Computer Science**

**By**

**Alqasam Ahmad Sona Allah Aljorn (163839)**

**Moath Atef Mostafa Abed (161328)**

**Mohamed Emad Deeb Al Khateeb (162376)**

**Ahmad Zahir Faiq Qasim (157830)**

**Fall 2025**

## نموذج حقوق الملكية الفكرية لمشاريع التخرج في قسم علوم الحاسوب

**يتم قراءة وتوقيع هذا النموذج من قبل الطلاب المسجلين لمشاريع التخرج في قسم علوم الحاسوب**

تعود حقوق الملكية الفكرية لمشاريع التخرج ونتائجها (مثل براءات الاختراع أو أي منتج قابل للتسويق) إلى جامعة العلوم والتكنولوجيا الأردنية، وتخضع هذه الحقوق إلى قوانين وأنظمة و تعليمات الجامعة المتعلقة بالملكية الفكرية وبراءات الاختراع. بناءا على ما سبق أوافق على ما يلي:

1) أن أحفظ كافة حقوق الملكية الفكرية لجامعة العلوم والتكنولوجيا الأردنية في مشروع التخرج.
2) أن ألتزم بوضع اسم جامعة العلوم والتكنولوجيا الأردنية و أسماء جميع الباحثين المشاركين في المشروع على أي نشرة علمية للمشروع كاملا أو لنتائجه. و يشمل ذلك النشر في المجلات و المؤتمرات العلمية عامة أو النشر على المواقع الإلكترونية أو براءات الاختراع أو المسابقات العلمية.
3) أن ألتزم بأسس حقوق التأليف المعتمدة في جامعة العلوم والتكنولوجيا الأردنية.
4) أن أقوم بإعلام الجهة المختصة في الجامعة عن أي اختراع أو اكتشاف قد ينتج عن هذا المشروع و أن ألتزم السرية التامة في ذلك و أن أعمل من خلال الجامعة على الحصول على براءة الاختراع التي قد تنتج عن هذا المشروع.
5) أن تكون جامعة العلوم والتكنولوجيا الأردنية هي المالك لأي براءة اختراع قد تنتج عن هذا المشروع و تشمل هذه الملكية حق الجامعة في إعطاء التراخيص و التسويق و البيع كمؤسسة راعية و داعمة لكافة الأنشطة البحثية. و يكون حق للطالب شمول اسمه على براءة الاختراع كأحد المخترعين، و في حال تم إعطاء تراخيص أو تسويق و بيع لأي من منتجات المشروع يمنح المخترعون بما فيهم الطالب نسبة من الإيرادات حسب تعليمات البحث العلمي في جامعة العلوم والتكنولوجيا الأردنية.

| | |
|---|---|
| إسم الطالب | التوقيع |
| إسم الطالب | التوقيع |
| إسم الطالب | التوقيع |
| إسم الطالب | التوقيع |
| **إسم المشرف** | **التوقيع** |

..... تاريخ

# ABSTRACT

In this digital era, the academic certificates fraud became a lot easier than ever, which makes the competition for opportunities unfair and the employers decision making harder too. That makes life more difficult for all the parties involved in the process. Right now, the process of verifying certificates is slow, manual chores. Because the structure of these systems is centralized, they are inefficient and the most important part is that they are prime target for cyberattacks, data breaches and manipulation. There is a glaring need for a verification method that is actually secure, globally accessible, private, and instant.

The core issue is that we currently lack a “gold standard” for instant verification. Instead, we rely on old procedures based on phone calls and emails. This causes a massive delay and lets a wide opened door for fraud. On the other hand, we can't forget that storing this sensitive data on centralized systems creates a single point of failure. If the database got hacked, the integrity of each certificate is compromised.

To solve this problem, we create Trustchain which is a blockchain-based system to modernize and enhance the process of handling academic certificates verification. When a university issued a certificate through Trustchain, it will be hashed, encrypted, and anchored to a decentralized ledger. Employees can use a web interface to verify the certificates instantly. This approach confirms the degree of authenticity with minimal personal data of student, effort and time from the verifier.

That results in showing that the system removes the vulnerability that comes from centralized authority; our system makes it impossible to edit and manipulate a certificate after it has been issued. On the other hand, the system turns weeks-long verification into an instant, one-click process.

Trustchain doesn’t just reduce fraud; it restores lost trust in the academic ecosystem.

**TABLE OF CONTENTS**

**PAGE**

**DEDICATIONS**.....#
**DECLARATIONS**.....#
**ACKNOWLEDGEMENTS**.....#
**ABSTRACT**.....#
**LIST OF TABLES**.....#
**LIST OF FIGURES**.....#
**ABBREVIATIONS**.....#

**CHAPTERS**
**1. INTRODUCTION**.....1
**1.1 OVERVIEW**.....1
**1.2 MOTIVATION**.....1
**1.2.1 INCREASE OF ACADEMIC CERTIFICATES FRAUD**
.....1
**1.2.2 THE INEFFICIENCY OF TRADITIONAL VERIFICATION METHODS** …… 2
**1.3 PROBLEM STATEMENT**.....2
**1.3.1 THE LACK OF SECURE AND RELIABLE VERIFICATION** …… 2
**1.3.2 ABSENCE OF GLOBAL VERIFICATION PLATFORM**
.....2
**1.4 SUMMARY OF THE FUNCTIONALITIES OF THE PROJECT**
.....2
**1.4.1 ISSUING CERTIFICATES USING BLOCKCHAIN** 2
**1.4.2 INSTANT AND SECURE VERIFICATION**.....3
**1.5 SUMMARY OF RESULTS**.....3
**1.5.1 IMPROVING TRUST AND TRANSPARENCY**.....3
**1.5.2 MAKING THE PROCESS MORE EFFICIENT**.....3
**1.5.3 REDUCE THE FRAUD**.....3
**2. REVIEW OF RELATED LITERATURE**.....4
**2.1. BLOCKCERTS** ……... 4
**2.1.1. OVERVIEW** …….. 4
**2.1.2. How it works** ……. 4
**2.1.3. Technology used** ……. 4
**2.1.4. Relation with our project** ……... 5
**2.2 OpenCerts** ……..5
**2.2.1 Overview** …….. 5
**2.2.2 Hot it works** …… 6
**2.2.3 Technology used** …….. 6
**2.2.4 Relation with our project** ……..... 6

**2.3 BCDiplma …… 6**
**2.3.1 Overview …… 6**
**2.3.2 How it works …….. 7**
**2.3.3 Technology used …….. 7**
**2.3.4 Relation with our project ……..... 8**

**3. ANALYSIS AND DESIGN …….. 9**
**3.1 System overview ……..…... 9**
**3.2 Diagrams …… 10**
**3.2.1 Use case diagram ……. 10**
**3.2.2 Sequence diagrams ……... 11**
**3.2.3 ER-diagram ……... 16**
**3.2.4 Class diagram …… 16**
**3.2.5 Activity diagram …… 19**

**4.IMPLEMENTATION .....21**
**4.1 Introduction ……. 21**
**4.2 Verification page …….. 21**
**4.2.1 Verification page …….. 21**
**4.2.2 Valid certificate example …….. 22**
**4.2.3 Invalid certificate example …….. 23**
**4.2.4 Revoked certificate example ……... 24**
**4.3 Login and register page ……. 25**
**4.3.1 Login page …….. 25**
**4.3.2 Registration university page ……... 26**
**4.4 Admin portal …… 27**
**4.4.1 Dashboard …… 27**
**4.4.2 Pending applications …… 27**
**4.4.3 Approve application popup ……. 28**
**4.4.4 Rejected application popup ……. 28**
**4.4.5 Registered universities ……. 29**
**4.4.6 Suspend university popup ……... 29**
**4.4.7 Reactivate university popup …… 30**
**4.5 University portal ……... 30**
**4.5.1 Issue certificate ……. 30**
**4.5.2 Issued certificates ……. 31**
**4.5.3 Valid certificates details ……... 31**
**4.5.4 Revoked certificate popup ……... 32**
**4.6 Technology used …… 32**
**4.6.1 FastAPI ……... 32**
**4.6.2 Polygon amoy testnet ……. 32**
**4.6.3 MongoDB+Motor ……... 32**
**4.6.4 Solidity^0.8.19 …… 32**

**4.6.5 Pinata (REST-API) …… 33**
**4.6.6 JWT …… 33**
**4.6.7Web3.py …….. 33**
**4.6.8 React.js ……..... 33**
**4.6.9 MetaMask …… 33**
**4.6.10 AES-256-GCM …….. 33**
**4.6.11 bcrypt (direct) ……... 33**
**4.6.12 SHA-256 ……. 33**
**4.6.13 qrcode+pyzbar …….. 33**
**4.6.14 univcorn server ……. 33**
**4.6.15 httpx ……... 33**
**4.6.16 pydantic-settings ……... 33**

**5.TESTING AND EVALUATION …… 34**
**5.1 Smart Contract (SC) Functional Testing …….. 34**
**5.2 User Acceptance Testing (UAT) …….. 35**
**5.3 Integration Testing (Backend API) ……... 36**

**6.CONCLUSION ……. 37**
**6.1 Conclusion and achievements ……... 37**
**6.1.1 Immutable records keeping ……... 37**
**6.1.2 Instant global verification …..……... 37**
**6.1.3 Cost-Effective Infrastructure ……... 37**
**6.2 Strengths and weaknesses ……. 37**
**6.2.1 Strengths ……..……..…….. 37**
**6.2.2 Weaknesses ……..……..…….. 37**

**REFERENCES ……... 38**
**APPENDICES …… 39**

**APPENDIX A: TITLE OF APPENDIX A**.....**39**
**APPENDIX B: TITLE OF APPENDIX B**.....**#**

# LIST OF TABLES

**TABLE** **PAGE**

**1.1 Title of The First Table in Chapter 1.....#**
**1.2 Title of The Second Table in Chapter 1.....#**
**1.3 Title of The Third Table in Chapter 1.....#**

**2.1 Title of The First Table in Chapter 2.....#**
**2.2 Title of The Second Table in Chapter 2.....#**

# LIST OF FIGURES

**FIGURE** **PAGE**

**1.1 Title of The First Figure in Chapter 1 ..... #**
**1.2 Title of The Second Figure in Chapter 1 ..... #**
**1.3 Title of The Third Figure in Chapter 1 ..... #**

**2.1 Title of The First Figure in Chapter 2 ..... #**
**2.2 Title of The Second Figure in Chapter 2 ..... #**

# CHAPTER 1

# INTRODUCTION

## 1.1 Overview

Considering the fast technological advancements witnessed worldwide in recent years, the digitalization of educational records has become an essential practice, especially within the higher education institution. This is because academic certificates play a crucial role in verifying the qualifications of the graduates and choosing between them in academic and professional environments. On the other hand, this technological advancement led to spread of academic certificates fraud, which in turn has undermined confidence in educational institutions and negatively affecting employers decision-making processes.

Traditional certificates issuance and verification procedures, which are based on centralized databases and manual verifications systems, are insufficient, time consuming, and vulnerable to manipulation and unauthorized access. So that led to dire need for secure, reliable, and scalable solutions that can ensure the CIA Triad of academic certificates.

This is where blockchain technology comes; it offers the desired approach to solve this problem due to its decentralized, immutable, and transparent characteristics.

## 1.2 Motivation

### 1.2.1 Increase of Academic Certificates Fraud

The incremental growth of online platforms has made the production and distribution of fake academic certificates easier. These fake credentials are frequently used to obtain employment opportunities in unlawful ways. This widespread fraud led to lack of trust in educational institutes and traditional verification procedures.

### 1.2.2 The Inefficiency of Traditional Verification methods

Currently, the used verification processes often involve manual communication between the institutions, employers, and third parties. These processes require significant time and effort, that leads to delay in hiring process and decision making. In addition, centralized systems have single points of failure and increase the risk of data breach and unauthorized modifications.

## 1.3 Problem Statement

### 1.3.1 The Lack of Secure and Reliable Verification

The existing certificates verification systems don't provide the needed protection against the forgery. Also, the centralized storage-based systems make the certificates vulnerable to data breaches and manipulation and cyberattacks, that affect the authority of the issued certificates.

### 1.3.2 Absence of Global Verification Platform

Currently, the world misses an accepted reliable global verification system. So, the employers rely on the manual ways to verify the certificates or third-party that provide verification services instead of depending on global instant system to do that. That increases the possibility of fraud and decreases efficiency too.

## 1.4 Summary of the Functionalities of the Project

### 1.4.1 Issuing Certificates using BlockChain

The proposed system (TrustChain) allows universities to issue their certificates over the blockchain technology which are securely stored hashed and encrypted. These certificates are recorded over the blockchain network that ensure immutability and resistance to manipulation.

### 1.4.2 Instant and Secure Verification

TrustChain allows the user meanwhile he is an employer, academic institute, etc. to verify authenticity of the certificate instantly using the web-based platform. However, it ensures that sensitive information is protected while providing reliable proof for the certificate using the records over the blockchain.

## 1.5 Summary of Results

### 1.5.1 Improving Trust and Transparency

TrustChain system shows that blockchain technology can enhance trust and transparency in academic certificate verification. That's because it didn't rely on centralized authorities, which ensure that the certificates won’t be replaced or edited after issuance.

### 1.5.2 Making the Process more Efficient

By implementing the proposed solution that will reduce the time and effort needed to verify the academic certificates, due to it is an instant process done on the web-based platform. Also, it did not require direct communication between the verifier and the issuer.

### 1.5.3 Reduce the Fraud

TrustChain system minimizes fraud because of its structure and decentralized storage model that increases the security and authenticity of the system.

# CHAPTER 2

# REVIEW OF RELATED LITERATURE

Because the idea does not belong to the one who says it, but to the one who implements it, we have put some efforts in searching , detecting , reading and discovering any similar person , company or organization which made a project like or pretty close to our idea and we found out something interesting :

## 2.1. Blockcerts

### 2.1.1. Overview

an open standard for creating, issuing, viewing, and verifying blockchain-based certificates. These digital records are registered on a blockchain, cryptographically signed, tamper-proof, and shareable. The goal is to enable a wave of innovation that gives individuals the capacity to possess and share their own official records,[1].

### 2.1.2. How it Works

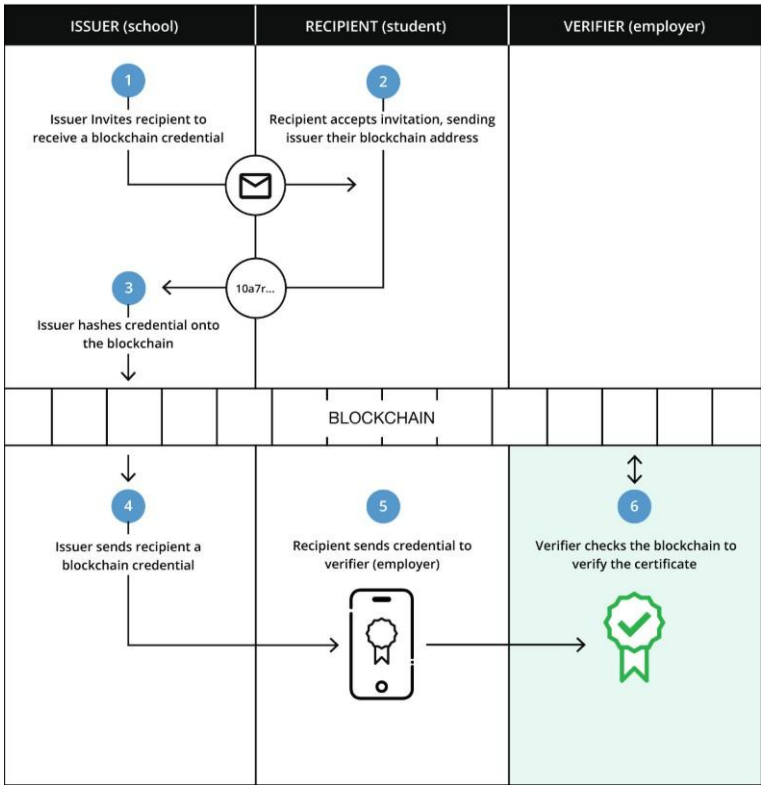



*Figure 2.1.2 Blockcerts*

### 2.1.3. Technology used

Blockchain certificate schemes implement the verifiable credentials data model. Aligned with these standard verifiable credentials, Blockcerts provided both a JSON-LD context and JSON schema [2]. The purpose of the JSON-LD

context is to map types to Internationalized Resource Identifiers (IRIs), providing semantic context for data. The JSON Schema is used for syntactic validation, After generating hashes for a selected batch of multiple JSON files , using the merkle-tree hash algorithm ,[3],the system produce one hash represents the number of certificates issued by the institution, this methodology known as merkle-proof,[4] verification.However,when using Blockcerts, the certificate holder must not lose his JSON file, otherwise it is lost forever ,[5].

#### 2.1.4. Relation with our project

While Blockcerts offers decentralized authenticity system for certificates their functions are more like tool-based rather than a real product beside the extensive technical infrastructure because it uses a highly cost blockchain solutions like ethereum blockchain,[6] which significant amount of fees known as gas-fees[7] , also it lacks the localization in our region ,preventing a lot of educational institution and organizations from having this layer of authentication.

## 2.2 OpenCerts

#### 2.2.1. Overview

Part of Singapore's Smart Nation initiative, enables issuance and validation of tamper-resistant academic certificates in a convenient and reliable manner, OpenCerts,[8] is built on top of OpenAttestation,[9]. One might be confused to understand the differences between OpenAttestation and OpenCerts:

- OpenAttestation is a standard that helps to create verifiable documents.
- OpenCerts is a specific implementation, using OpenAttestation, whose purpose is to provide institutions with a way to create verifiable certificates.

### 2.2.2. How it Works

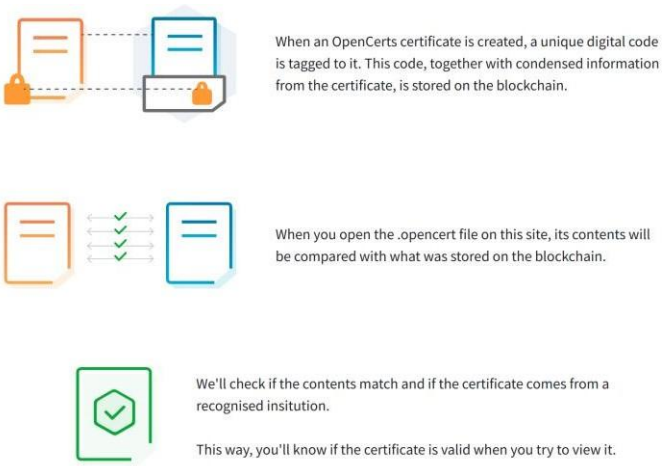



*Figure 2.2.2 Opencerts*

### 2.2.3. Technology used

OpenCerts has a various similarities with BlockCerts such as utilizing the ethereum blockchain. However, this methodology has solved an important question: "*How can I ensure the validity of this certificate issuer?".* This is achieved by using DNS-TXT,[10] method, which binds the issuer's DNS connected with the wallet that produces the transaction.

### 2.2.4. Relation with our project

While OpenCerts is a government-based project similar to our goal, but it has major issues, if the student loses the .opencert file, the credential is lost forever. However, unlike our goal of cost efficiency, relying on ethereum mainnet costs high gas fees [6], besides the inability to modify the DNS records. Finally, being a relatively older project, it relies on legacy architectural choices compared to modern layer-2 solutions [11]. For these reasons, our project prioritizes sustainability, user experience, and cost efficiency.

## 2.3. BCDiploma

### 2.3.1. Overview

a successful commercial company based in France, provides a variety of digital authentication services using blockchain [12]. Their philosophy came up to solve a rising question in the two previous projects above: "What if the certificate holder lost his certificate proof?" [13] By creating a URL link which anyone can view on the blockchain [13], if anyone wants to verify his certificate, he can just click on the link, and it will show his digital certificate. The detailed process will be discussed below.

### 2.3.2. How it works




*Figure 2.3.2 BCDiploma*

### 2.3.3. Technology used

The project development team have put their efforts in building a sustainable ecosystem called “Evidenz” [14], with blockchain architecture and a payment method using their token “BCDT” [15] which keeps the system alive, the workflow presents in these 4 steps

- **2.3.3.1. Encryption**

  The certificate issuer enters the data in the system, the AES-256 encryption algorithm is applied to this data, and a decryption key is generated specifically for this certificate [16].

- **2.3.3.2. On-Chain Storage**

  The encrypted data will be sent only to be stored inside a smart contract on the ethereum blockchain [6]. For security reasons, the decryption key is not stored on the blockchain ever.

- **2.3.3.3. The Smart Link**

  The system creates a URL, which contains the decryption key for the certificate [14].

- **2.3.3.4. Rendering**

  When the link is pressed, the browser reads the decryption key from the link, requests the encrypted data from the blockchain, then it will be encrypted locally on the same browser, previewing the certificate [14].

### 2.3.4. Relation with our project

While BCDiploma offers a wide range of authentication services [12], utilizing a custom-build framework imposing a significant operational burden, paying a high gas-fees for using Ethereum blockchain [6]. In contrast, our project prioritizes optimal cost efficiency, user-friendly, platform, and secure credentials.

# CHAPTER 3

# ANALYSIS AND DESIGN

In our system, before starting the design phase, we need to perform the analysis phase to determine the requirements, which are the main output of the analysis phase, and define the functionality of our system. The technologies we will use are Blockchain and Web 3 to validate the certificates. After the analysis phase, we are now ready to move to the design phase, in which we need several types of diagrams such as ER Diagram, Sequence Diagram, Use Case Diagram, Class Diagram, and each of them for a specific function. These diagrams are used in the implementation phase.

## 3.1 System overview

The system is designed so that each university on this system has a public key (wallet address), and university degrees from the same university are linked to this public key. Each degree has a unique hash number and QR-code. Then, I read this QR-code or upload the PDF degree file. If the degree is original, it will show that it is available. However, if it has been modified by the university and this version has been canceled, or if there was a mistake and a new version was issued, it will show that it is revoked. Finally, if you don't have any degree or the QR code or the PDF you entered is incorrect, it will show that it doesn't exist.

## 3.2 Diagrams

### 3.2.1 Use Case diagram

In the use case diagram, we have three main actors: Admin, University, and Verifier. Let's break them down one by one. The first actor is the admin, who has the highest authority. Their roles are as follows: Suspend University, which occurs when the issuance of certificates from a university is stopped for specific reasons, or Register University, where their role is to identify the university and authorize it to upload certificates under its name, thus attributing the certificates to that university, and Manages Universities access to the system by Suspend after Register it to the system or vice versa, and view all Universities on the system. Next actor is the University, which represents the entity that issues these certificates and uploads them to the system. Their roles are: Issue Certificate, which is done by taking the student's information, attaching it to the certificates, and then uploading it to the system as if the certificate is accredited by the university; and Revoke Certificate, which happens when a student modifies the information provided by the university, or if there is an error in the certificate. However, note that if a certificate is canceled, it is not deleted from the system. It will be marked as excluded and invalid, and Manages Certificates on the system by Revoke it after Register it to the system or vice versa, and view all Certificates on the system. The last actor is the Verifier, whether the employer is an academic institution, a business, or any other entity that wishes to verify the certificate, if it is exist and valid, excluded or does not exist as all that done by scanning QR-code on the certificate or Uploading PDF degree file, and then gives the result instantly whether it was available or revoke or fake.

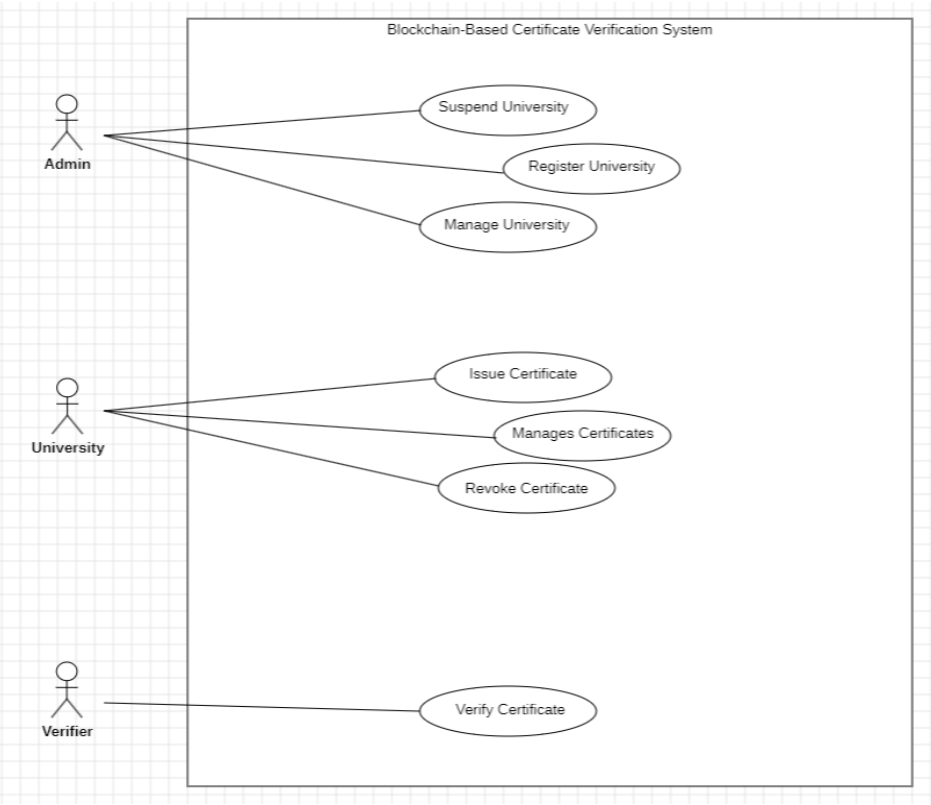



*Figure 3.2.1 Use Case*

## 3.2.2 Sequence diagrams

### 3.2.2.1 Issuing Certificate Sequence diagram

When university have the certificate data, we send it to Issue Certificate after this, send Certificate data and PDF to Backend API to validate data and generate Hashed Certificate and encrypt PDF, then send them to IPFS to store Encrypted PDF and generate CID, then send the CID to Backend API to Encrypt CID and generate QR Code and then save Certificate status to be 'Pending', then return Certificate ID, Certificate Hash, Encrypted CID and QR Code to Issue Certificate to send request to University Wallet for signature, after that issue Certificate with Certificate Hash and Encrypted CID in Smart contract and store metadata, then return the Transaction Hash to Issue Certificate, after that send Transaction Hash to Backend API to verify Transaction Receipt and update status to Issued in Backend API and send it to Issue Certificate, and finally send a success message to University.

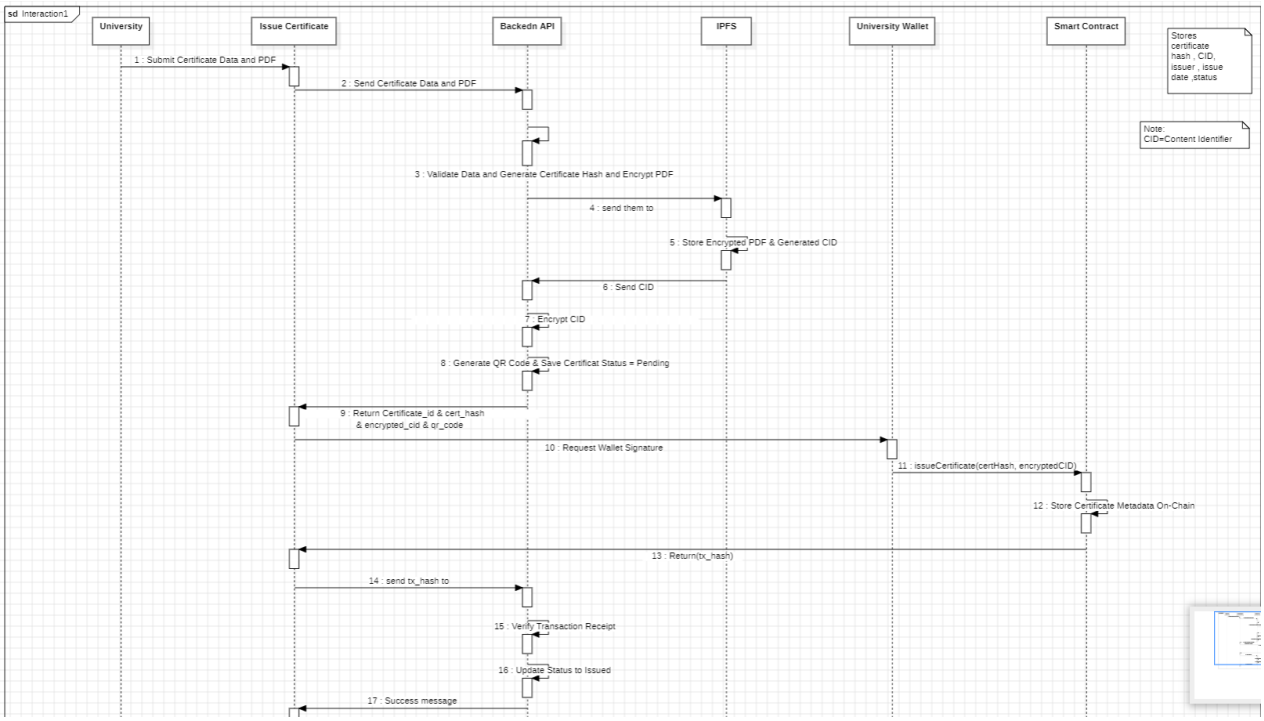



*Figure 3.2.2.1 Issue Certificate*

### 3.2.2.2 University Registering Sequence diagram

When admin receive a registering request (application), we display in pending applications the registering requests, after that we approve and send it to backend API, then send this request to database to create a university record in the university collection and remove from application collection, and finally we remove the registering request from the pending applications and add it to registered universities.

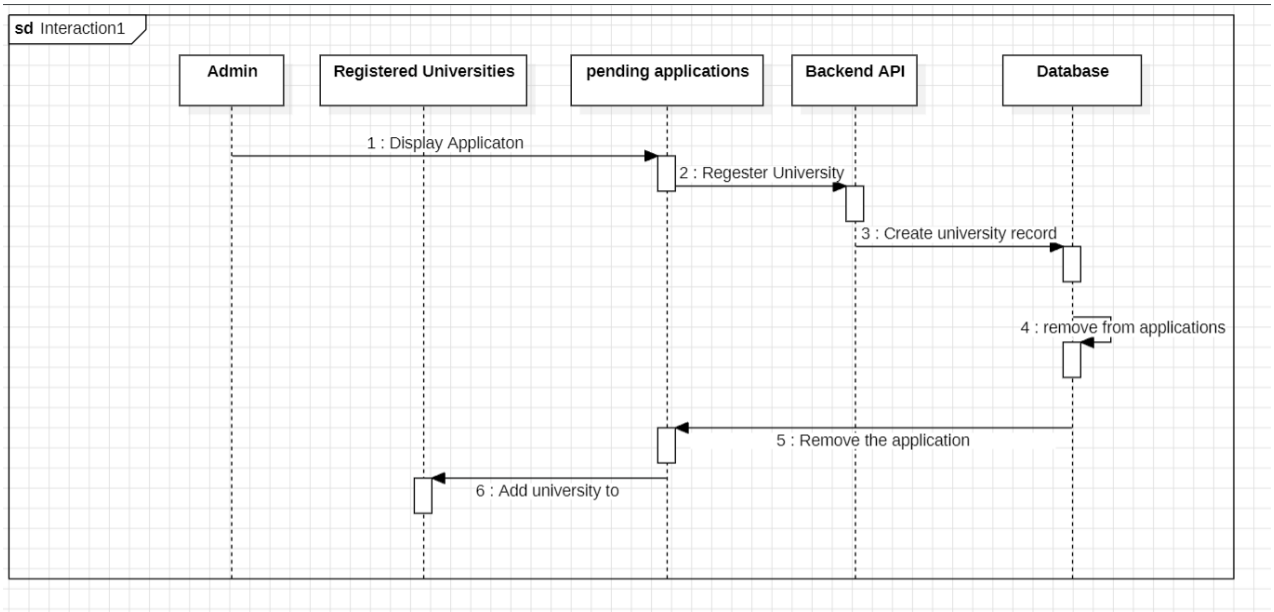



*Figure 3.2.2.2 University Register*

### 3.2.2.3 University Suspension Sequence

When admin receive a registering request, we display in pending applications the registering requests, after that we suspend that request and send it to backend API, then send a request to database to edit the status to rejected in application collection without delete it, and finally remove the application from the pending applications.

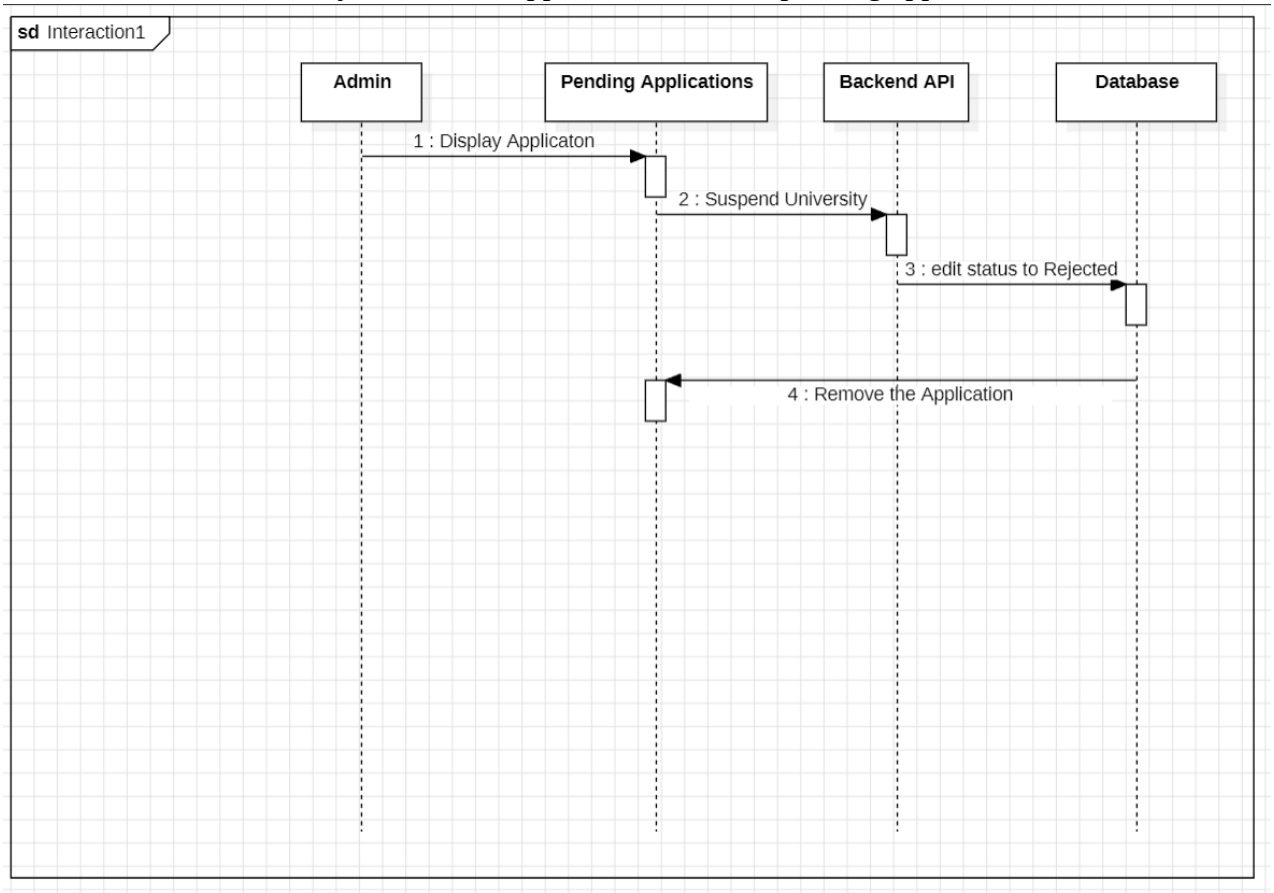



*Figure 3.2.2.3 University Suspention*

### 3.2.2.4 University Registration Request Sequence

When university make a registering request, we send this request from registration page to backend API after check if the password and the confirm password is the same, then we check if this data belong to any application or university in the database collections, after that we send a knowledge message from database to backend API, finally we send a success message to registration page and add this request in pending applications in the admin.

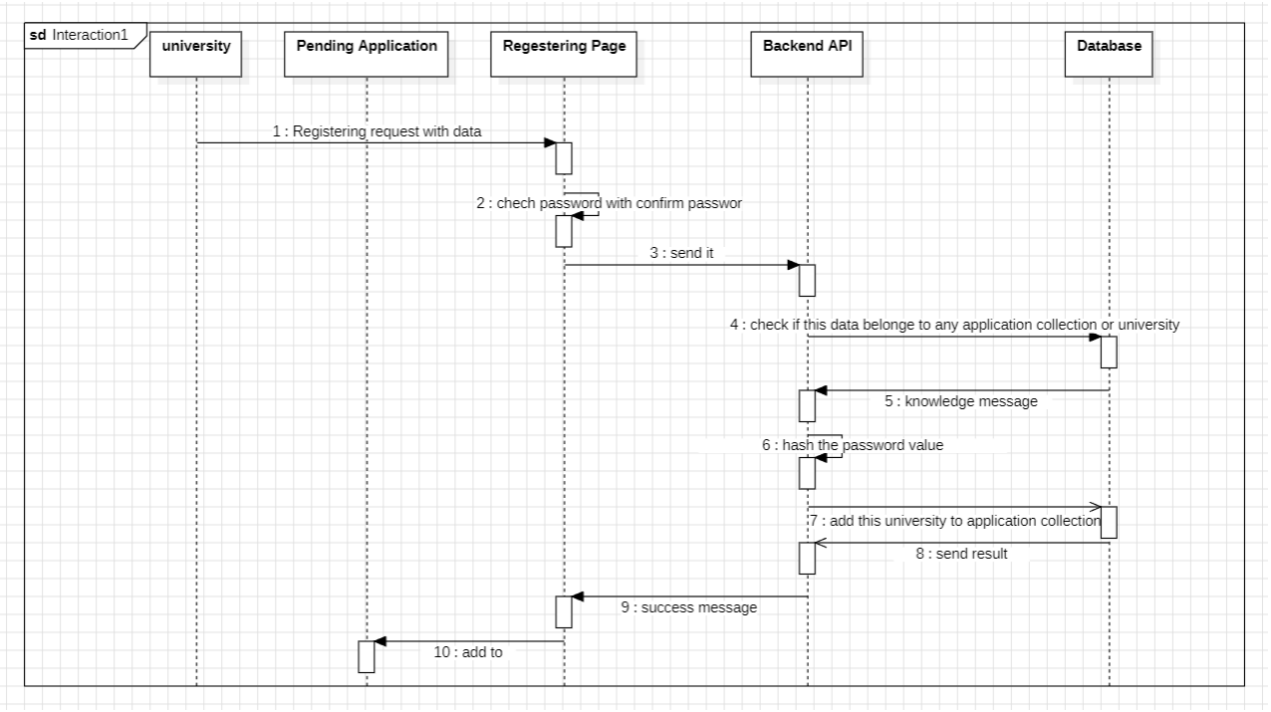



*Figure 3.2.2.4 University Registration Request*

### 3.2.2.5 Verifier Sequence diagram

When verifier want to verifying the certificate, first verifier scans QR-code or upload PDF degree file in verification page, then verification page sends certificate file and CID to backend API, in backend API generate hash and save it temporary, then send CID to smart contract to retrieve package certificate, then return certificate package to backend API, then backend API compare saved hashed with the generated one, finally display the verification result in verification page.

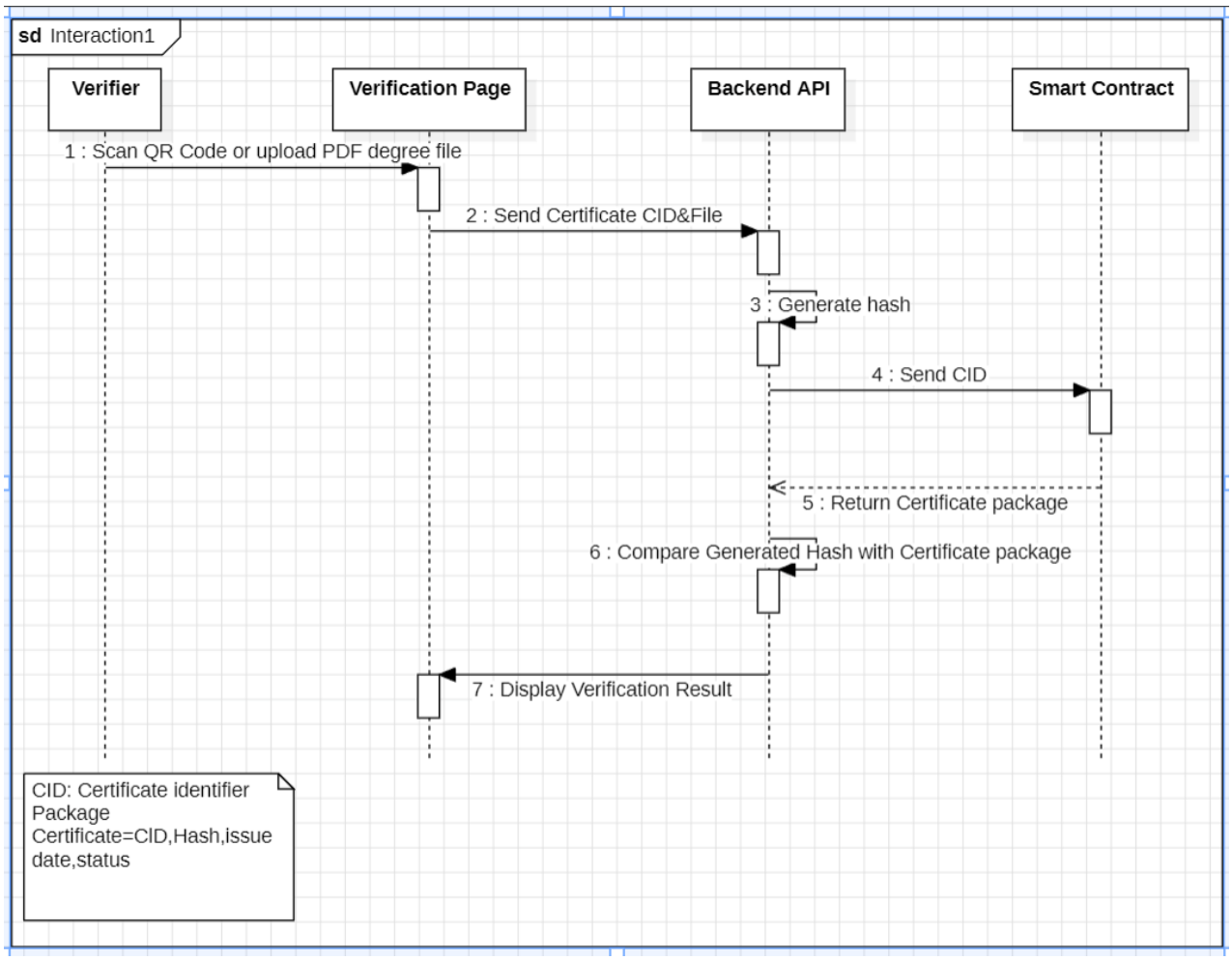



*Figure 3.2.2.5 Verifier diagram*

### 3.2.3 ER-diagram

In this diagram we have 4 entities (Admin, University, Applciation, Certificate), first entity Admin which have 5 attributes (_id, email, hashed_password) and one relation (manage) with university entity (mandatory one to mandatory many), second entity University which have 9 attributes (_id, university_name, country, email, wallet_address, total_certificate, status, registered_date, hashed_password) and two relations one with Admin entity (manage) and the second one (issues) with Certificate entity (mandatory one to mandatory many), third entity Application which have 7 attributes (_id, university_name, country, email, status, registered_date, hashed_password) and one relations (convert_to) with university entity(mandatory one to mandatory one), forth entity Certificate which have 5 attributes (ipfs_cid, certificate_hash, isuue_date, status, qr-code) and one relation with University (issues).

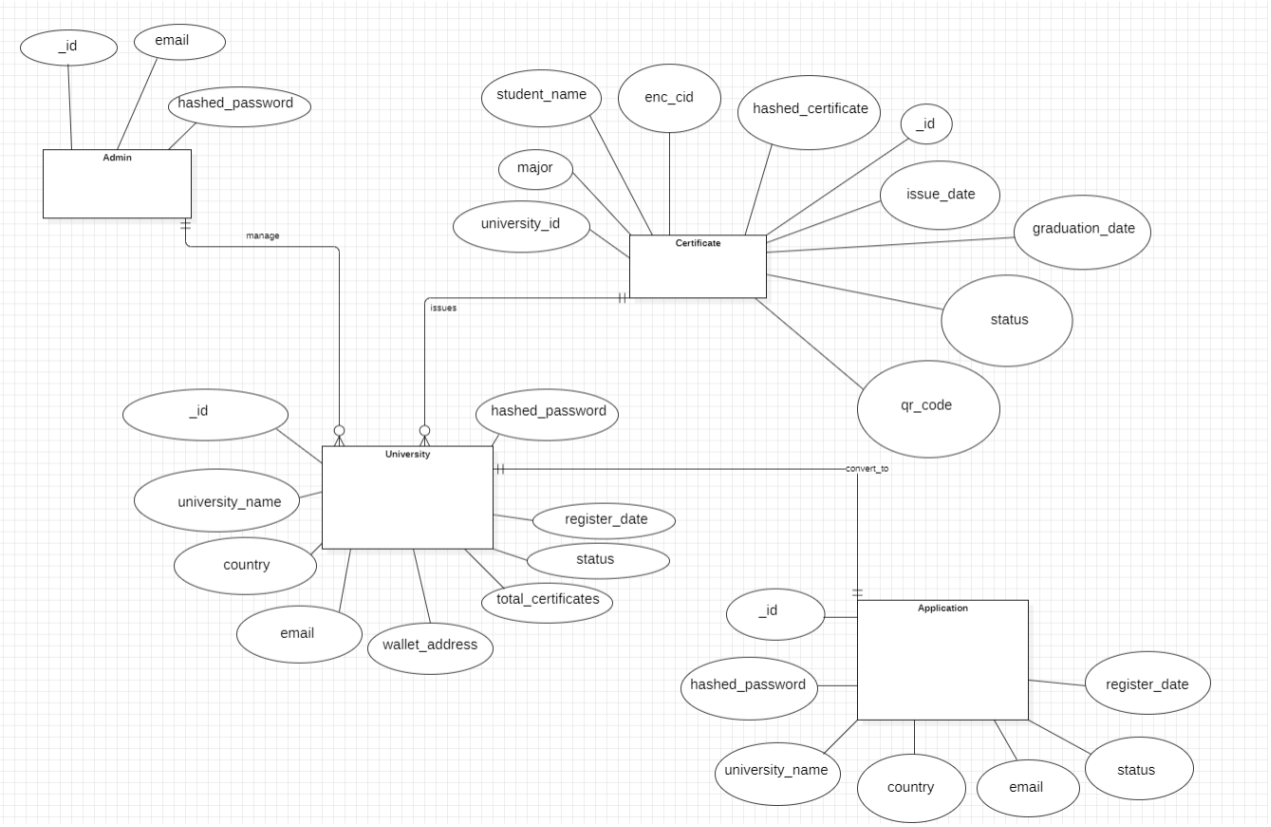



*Figure 3.2.3 ER-diagram*

### 3.2.4 Class diagram

The class diagram consists of seven main classes, each responsible for a specific role within the system. The following sections describe each class along with its attributes and operations.

#### 3.2.4.1 Admin Class

The admin class is responsible for managing and controlling the system. It handles administrative tasks related to university registration and status management.

##### 3.2.4.1.1 Attributes

_id
email
hashed_password

##### 3.2.4.1.2 Operations

registerUniversity()
suspendUniversity()

#### 3.2.4.2 University Class

The University class represents an authorized institution that is allowed to issue and revoke certificates within the system.

##### 3.2.4.2.1 Attributes

_id
university_name
country
email
wallet_address
total_certificate
status
register_date
hashed_password

##### 3.2.4.2.2 Operations

registerUniversityApplication()
registerWallet()
issueCertificate()
revokeCertificate()
manageCertificate()

#### 3.2.4.3 Certificate Class

The Certificate class represents the digital certificate and its associated data stored within the system.

##### 3.2.4.3.1 Attributes

hashed_certificate
enc_cid
Issue_date
status
qr_code
_id
student_name
major
graduation_date
university_id

#### 3.2.4.4 BackendService Class

The BackendService class acts as the core logic layer of the system. It handles validation, processing, encryption, and verification of certificate data.

##### 3.2.4.4.1 Operations

validateCertificateData()
generateCertificateHash()
storeCertificate()
verifyCertificate()
encryptCid()
decryptCid()

#### 3.2.4.5 IPFSService Class

This class is responsible for storing and retrieving certificate files in a decentralized manner using IPFS.

##### 3.2.4.5.1 Operations

uploadFile()
getFile()
genCid()

#### 3.2.4.6 BlockchainService Class

The BlockchainService class is responsible for storing and retrieving certificate metadata and hash values on the blockchain.

##### 3.2.4.6.1 Operations

storeCertificateMetadata()
getCertificateHash()

#### 3.2.4.7 Verifier Class

The Verifier class represents external entities or users who need to verify the authenticity of a certificate.

##### 3.2.4.7.1 Operation

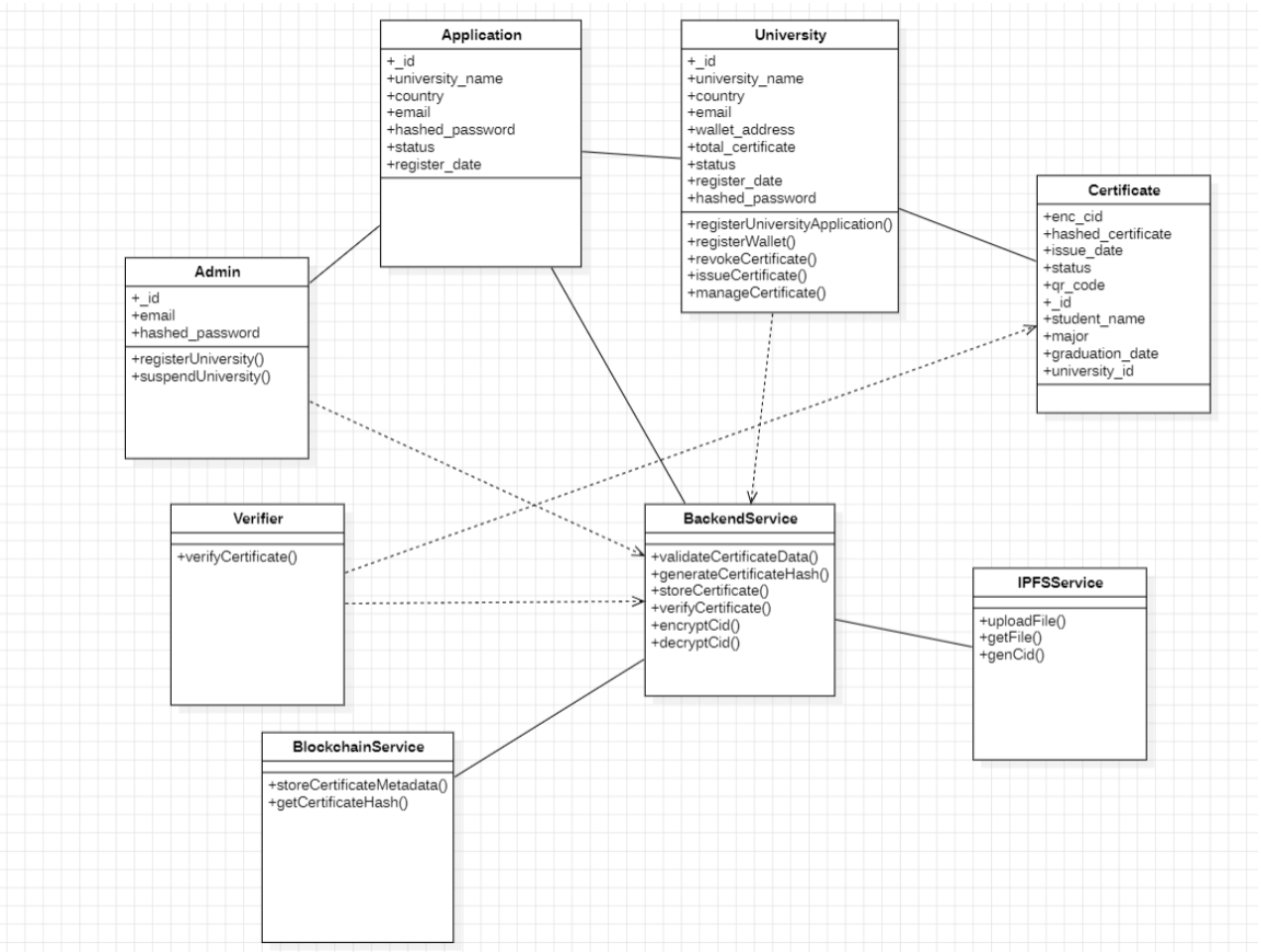
verifyCertificate()

#### 3.2.4.8 Application

The Application class is represent the new register university request before we register this university in our system.

##### 3.2.4.8.1 Attributes

_id
university_name
country
email
hashed_password
status
register_date




## 3.2.5 Activity Diagram

The activity diagram illustrates the workflow of the certificate issuance process. It is divided into five swimlanes, each representing a distinct entity involved in the data flow, from submission to blockchain anchoring.

#### 3.2.5.1 University Swimlane

The University swimlane represents the authorized user (staff or admin) who initiates the certificate creation process.

**3.2.5.1.1 Activities**

Submit Certificate Data

#### 3.2.5.2 University Portal Swimlane

The University Portal swimlane acts as the user interface (Frontend) that bridges the human user with the backend system.

**3.2.5.2.1 Activities**

Send Certificate Data

Send QR Code & Success Message

#### 3.2.5.3 Backend API Swimlane

The Backend API swimlane contains the core business logic. It handles

data validation, encryption, and coordinates communication between IPFS and the Blockchain.

**3.2.5.3.1 Activities**

Validate Data & Generate Hash
Encrypt Certificate
Package Certificate
Receive Acknowledgement
Send CID
Encrypt CID
Generate QR Code Encrypted CID

### 3.2.5.4 IPFS Swimlane

The IPFS swimlane is responsible for the decentralized storage of the physical certificate files to ensure availability and reduce blockchain load.

**3.2.5.4.1 Activities**

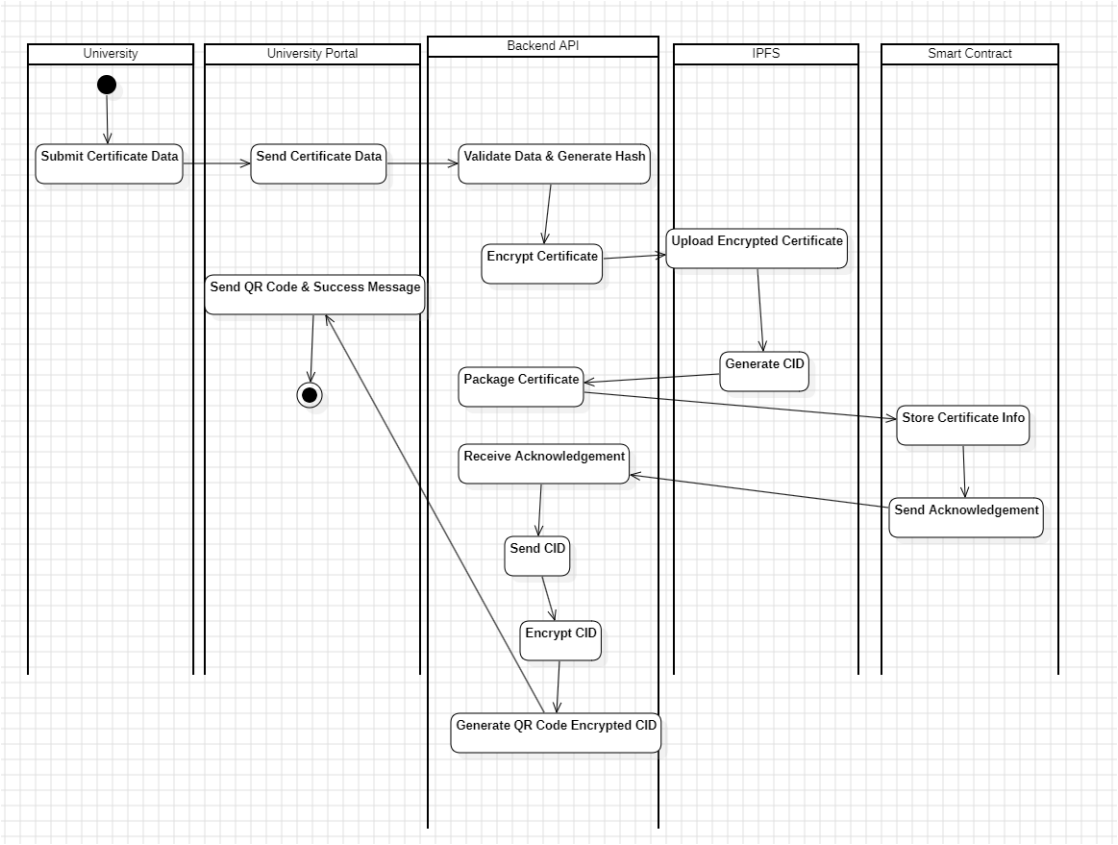
Upload Encrypted Certificate
Generate CID

### 3.2.5.5 Smart Contract Swimlane

The Smart Contract swimlane represents the immutable logic on the blockchain. It ensures that the certificate metadata is permanently recorded and verifiable.

**3.2.5.5.1 Activities**

Store Certificate Info
Send Acknowledgement



# CHAPTER 4
# IMPLEMENTATION

## 4.1 Introduction

This chapter presents the implementation details of Trustchain system by showing some figures of the project focusing on UI of the main pages which are verify page and example of different certificates status output as shown in 4.2.X figures, login and university register pages as shown in 4.3.X figures, admin portal that shows the registered universities and action can be done on it and the registration application received and its action too as illustrated in 4.4.X figures, and the main portal which is university portal where university can issue certificates and check its issued ones as demonstrated in 4.5.X figures.

## 4.2 Verification page

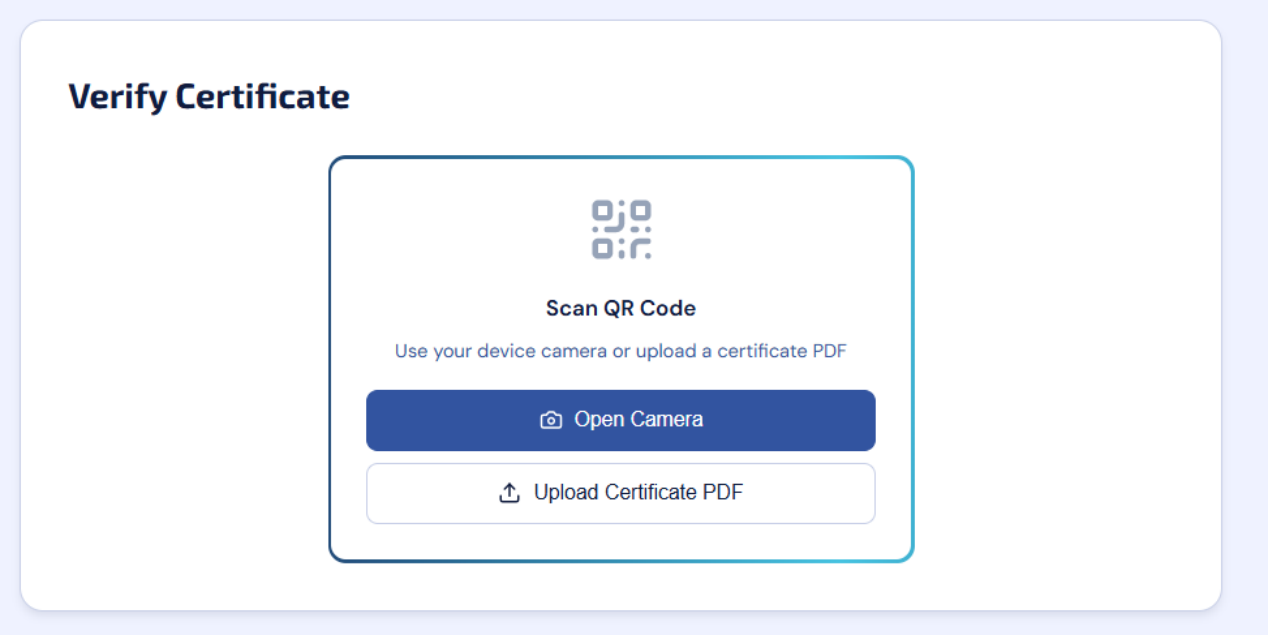



*Figure 4.2.1 Verification page*

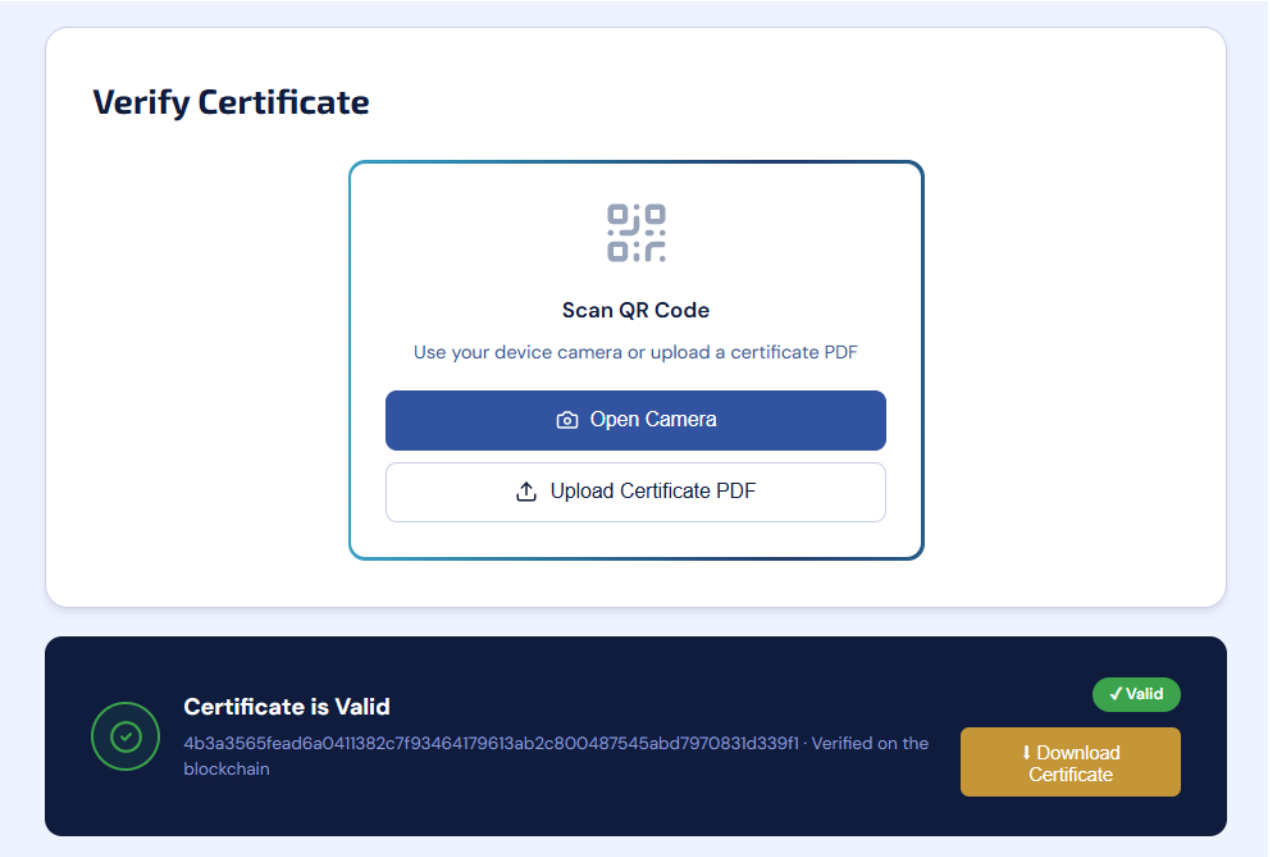



*figure 4.2.2 valid certificate example*

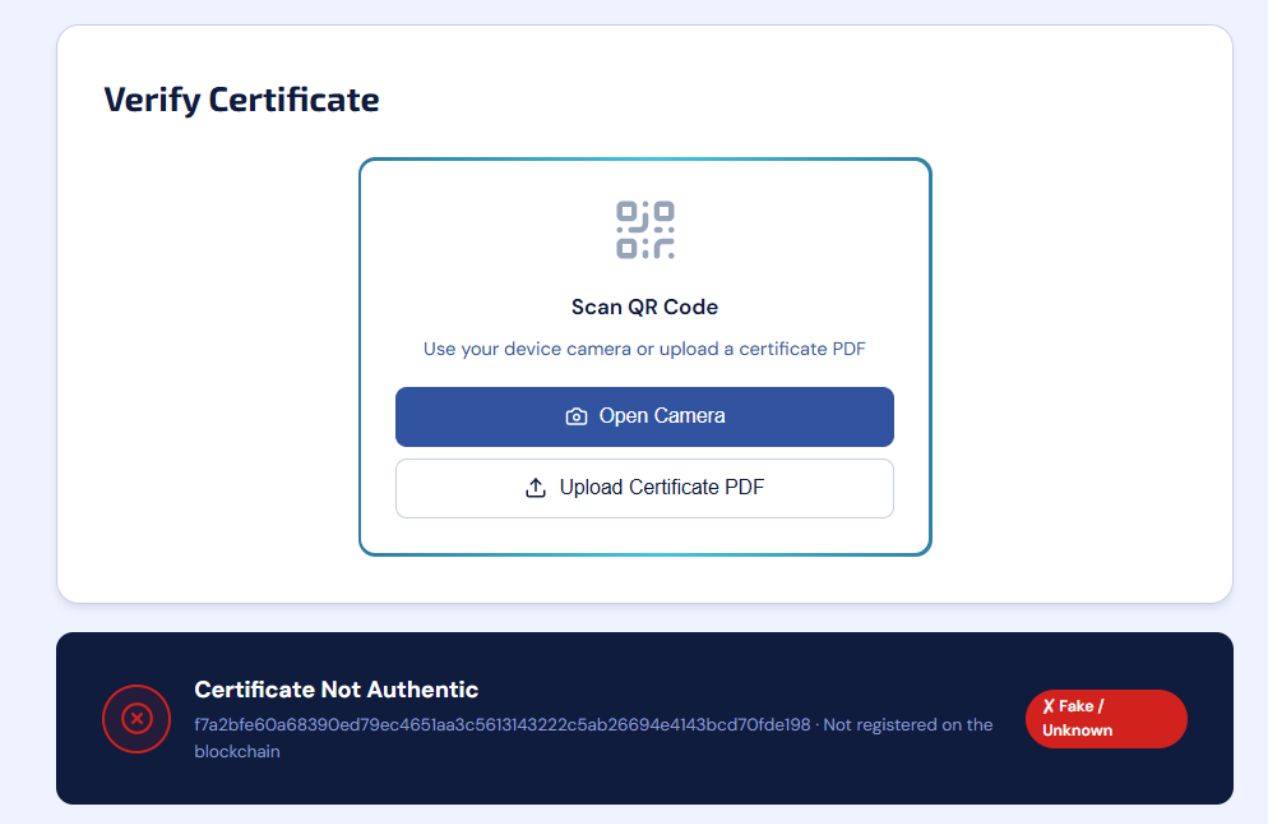



*figure 4.2.3 invalid certificate example*

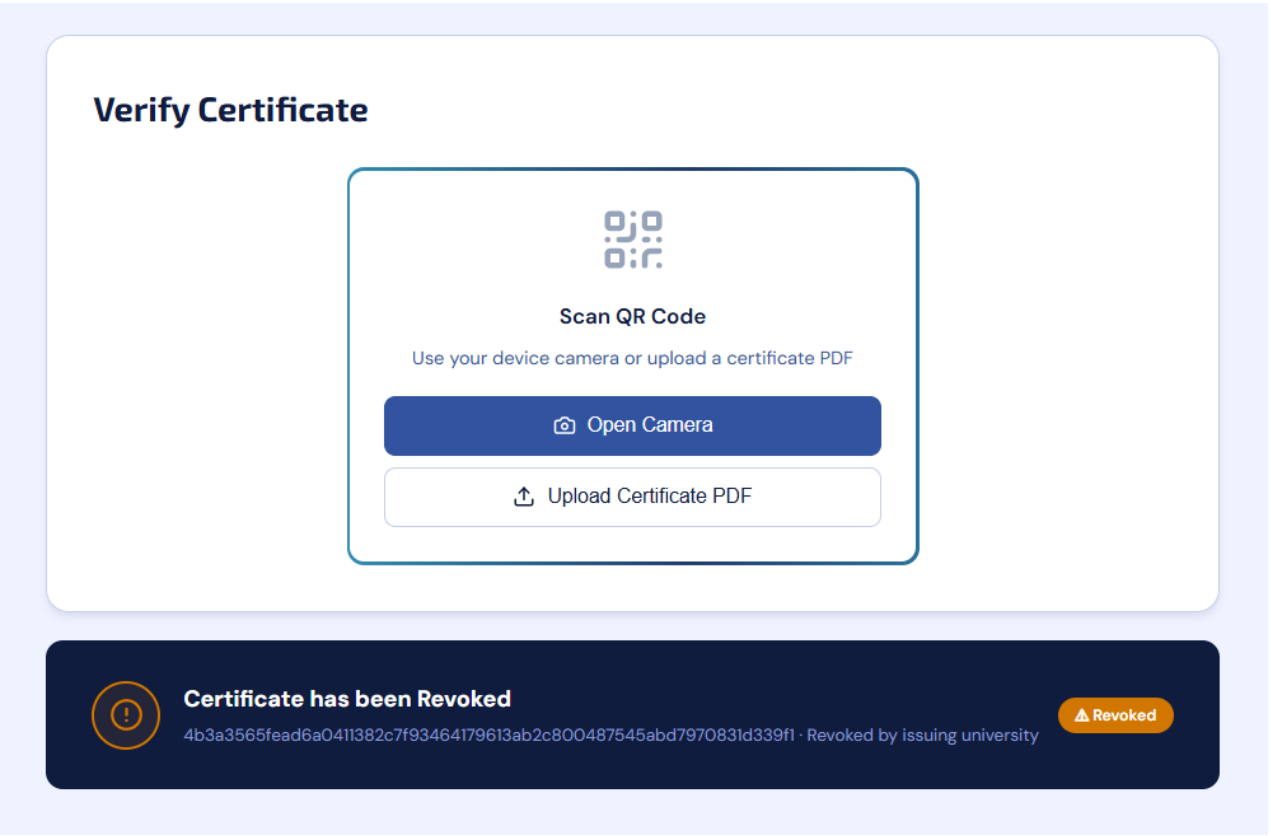



*figure 4.2.4 Revoked certificate example*

## 4.3 Login and register page

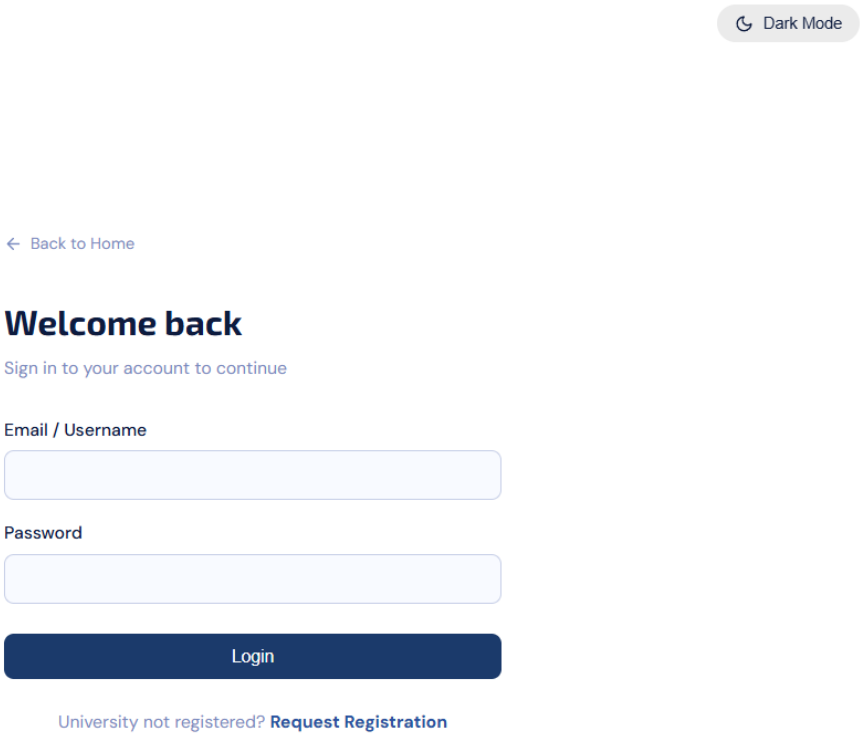



*figure 4.3.1 login page*

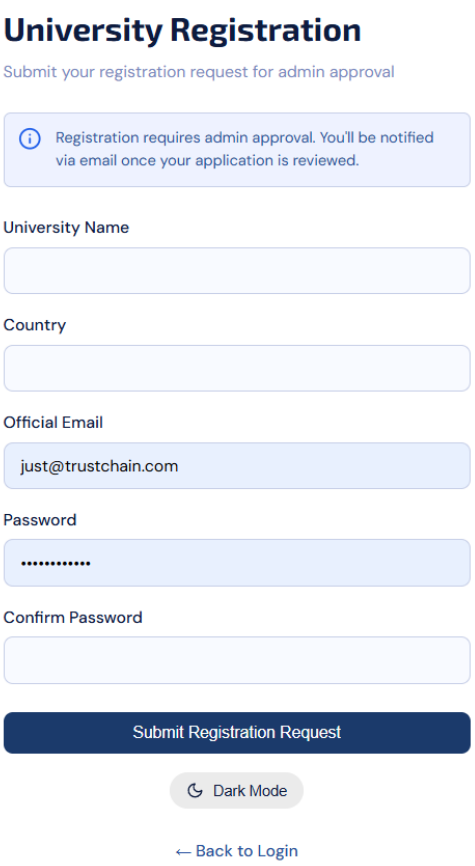



*Figure 4.3.2 Register university page*

## 4.4 Admin portal

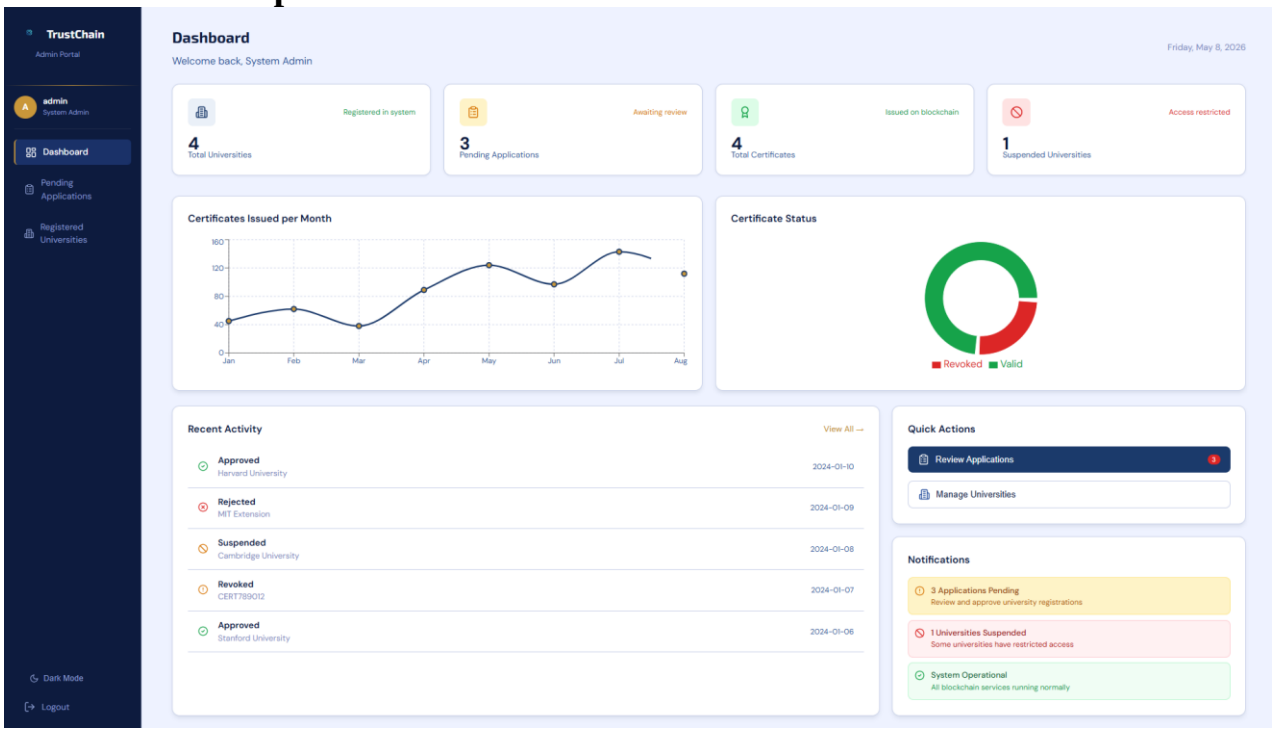



*Figure 4.4.1 Dashboard*

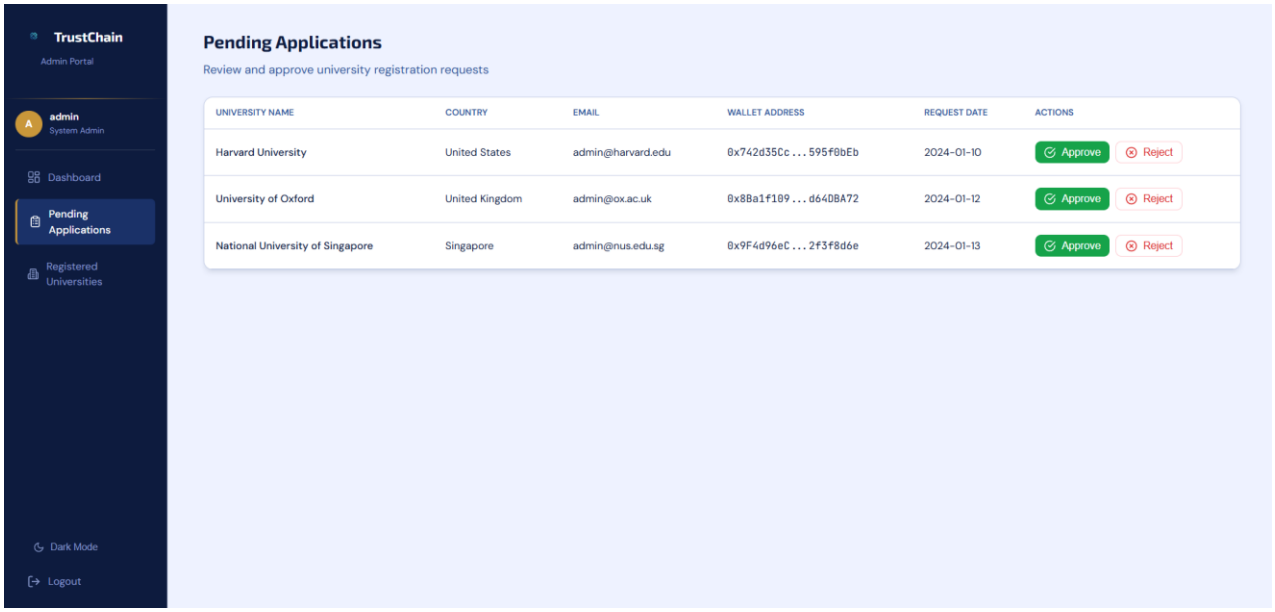



*Figure 4.4.2 Pending applications*

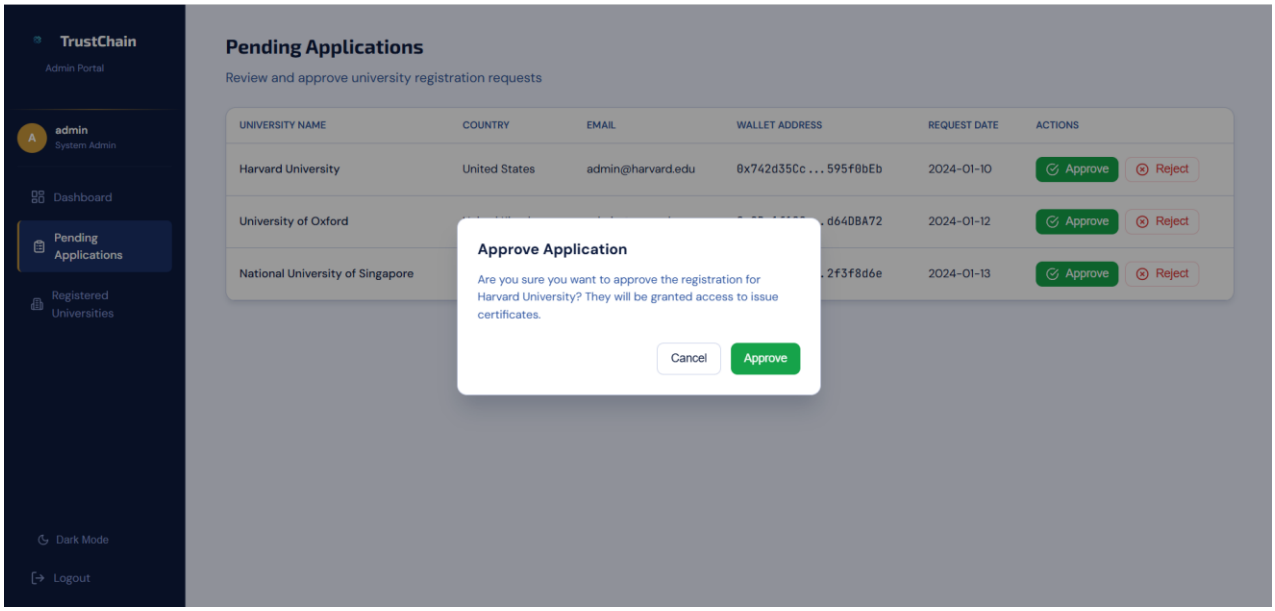



*Figure 4.4.3 Approve application popup*

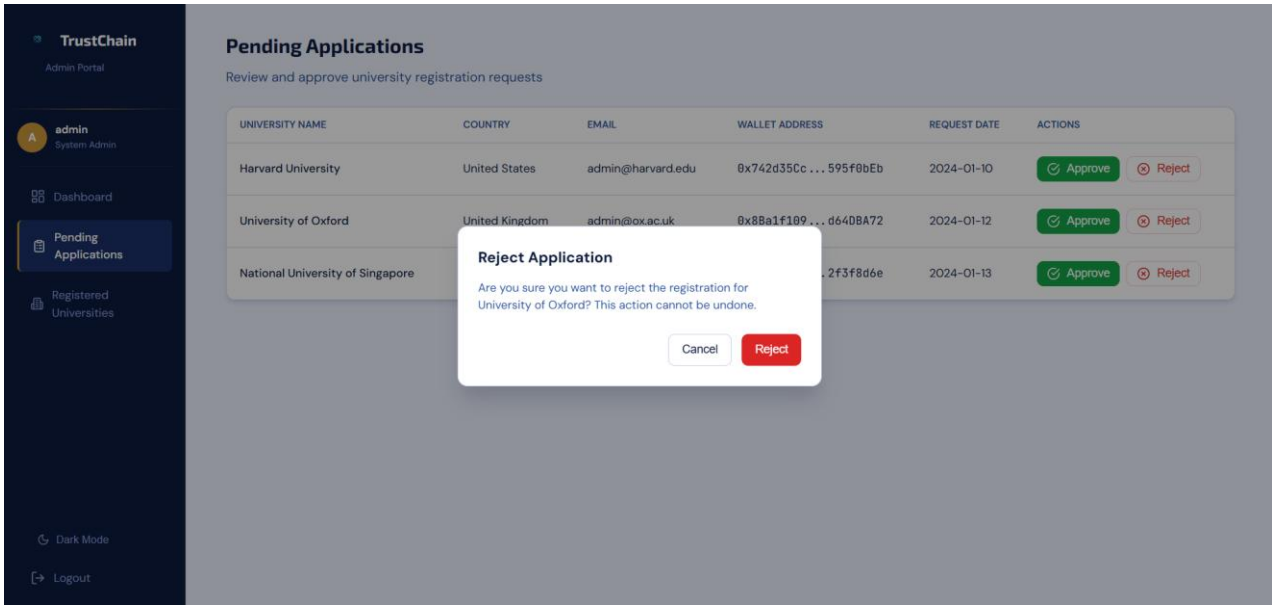



*Figure 4.4.4 Reject application popup*

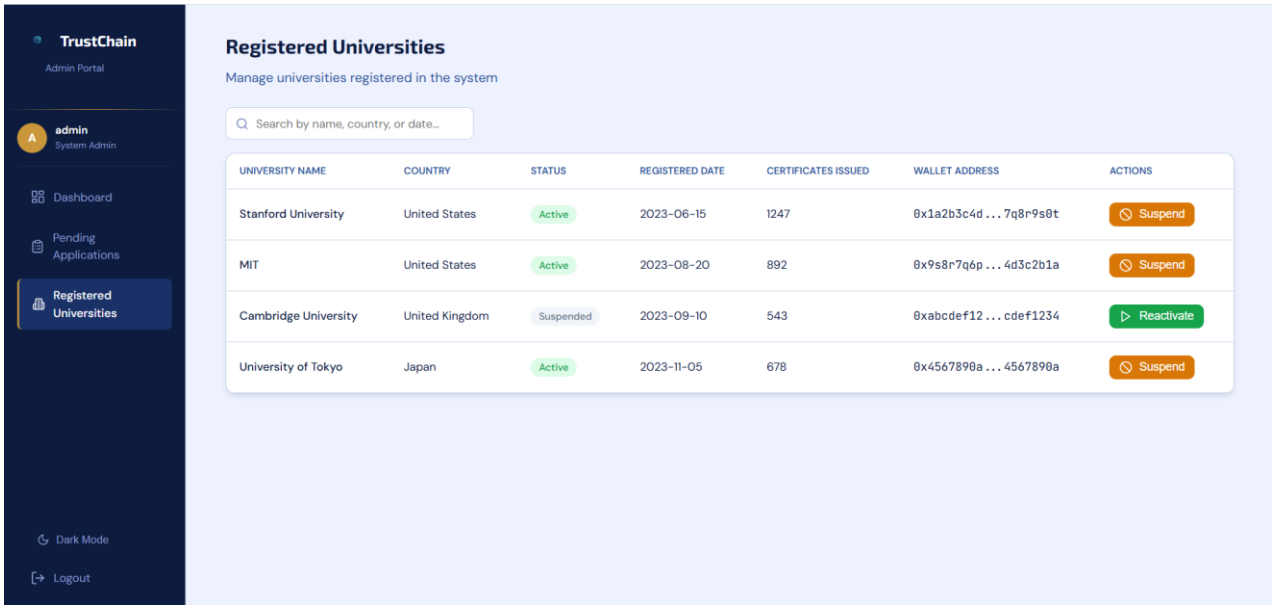



*Figure 4.4.5 Registered universities*

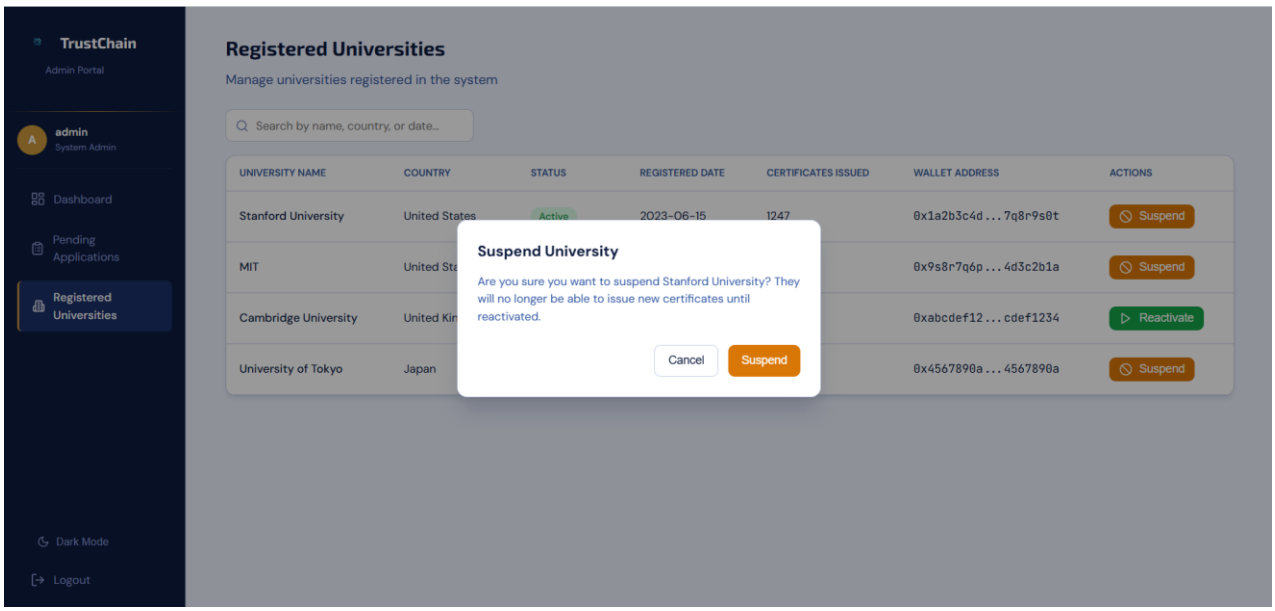



*Figure 4.4.6 Suspend university pop-up*

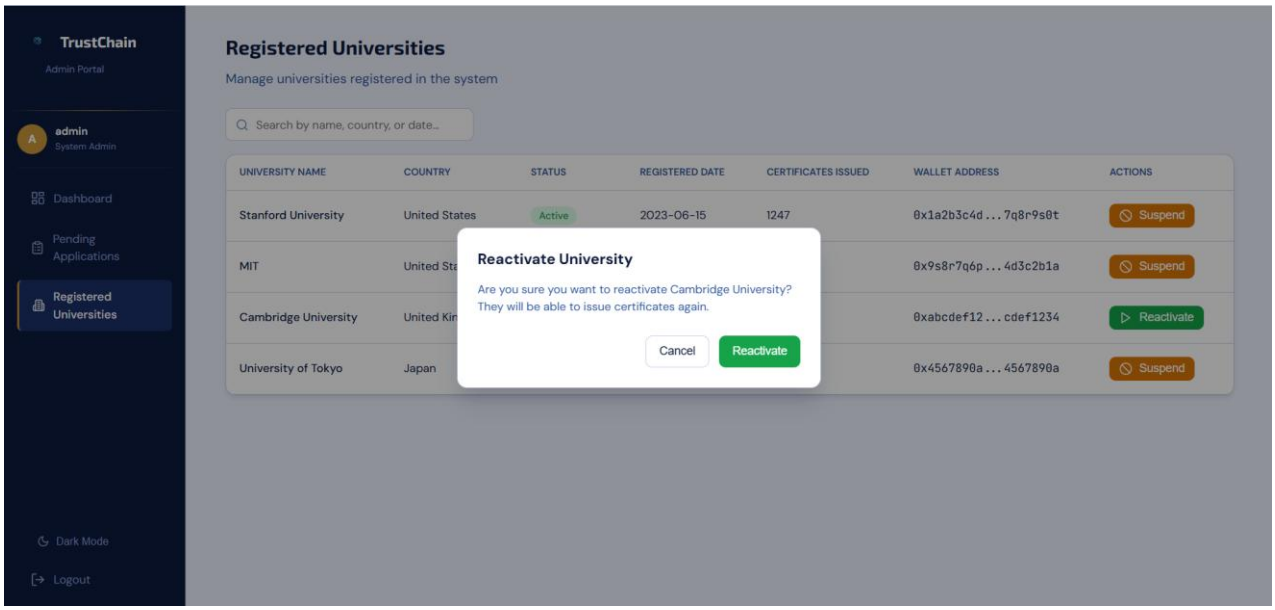



*Figure 4.4.7 Reactivate university popup*

## 4.5 University portal

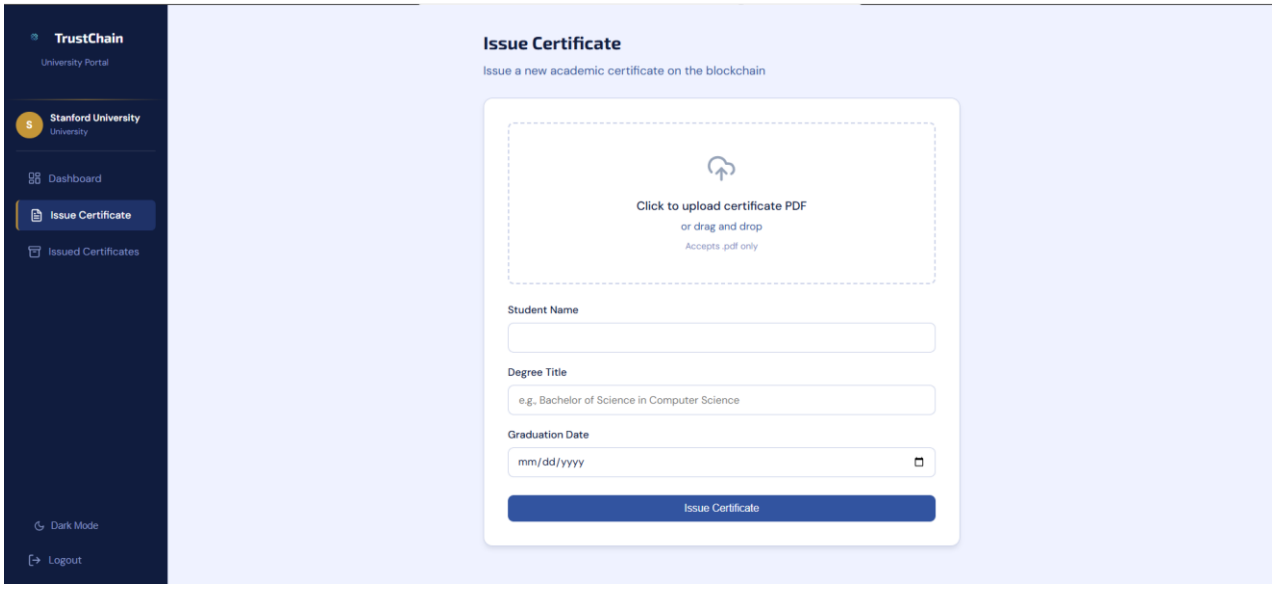



*Figure 4.5.1 Issue certificate*

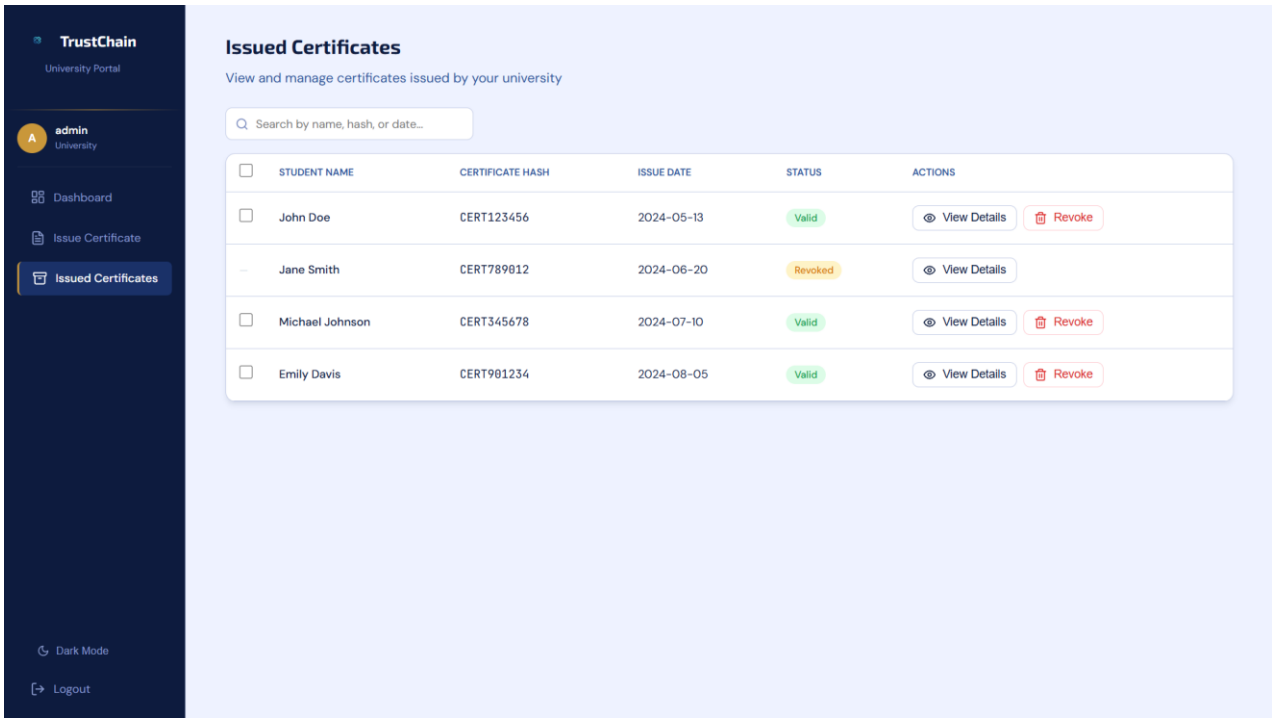



*Figure 4.5.2 Issued certificates*

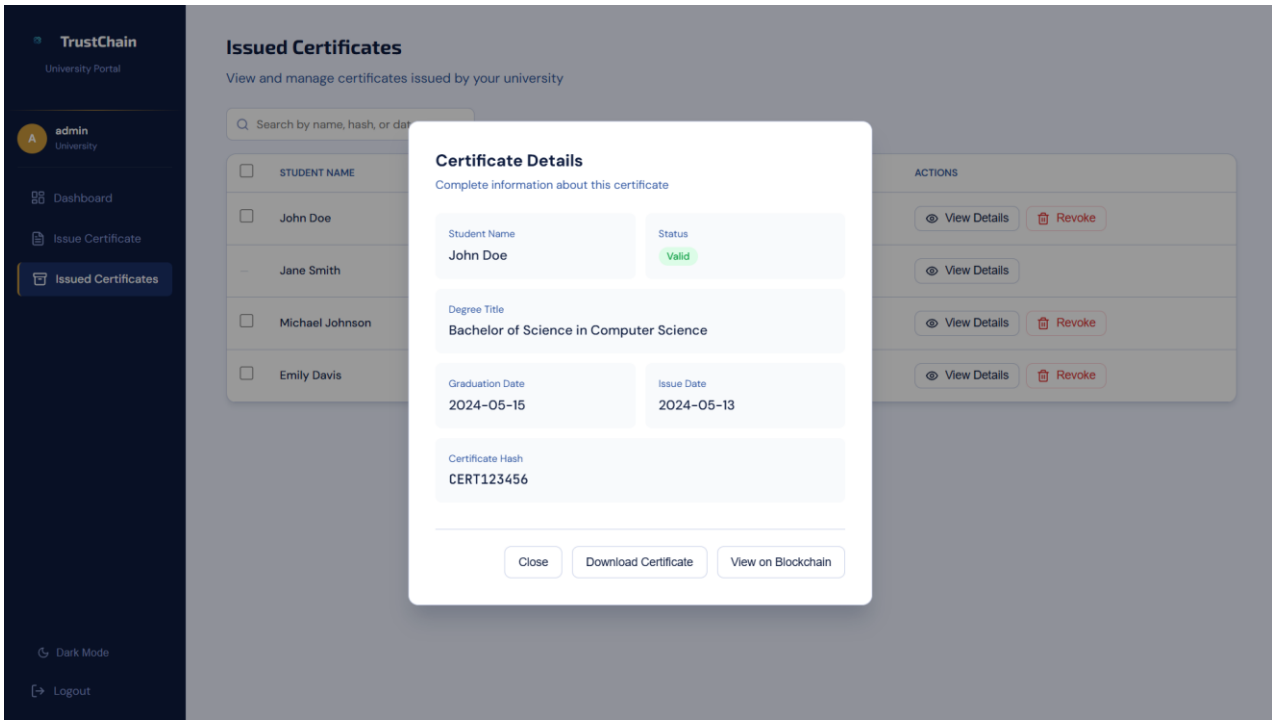



*Figure 4.5.3 Valid certificate details*

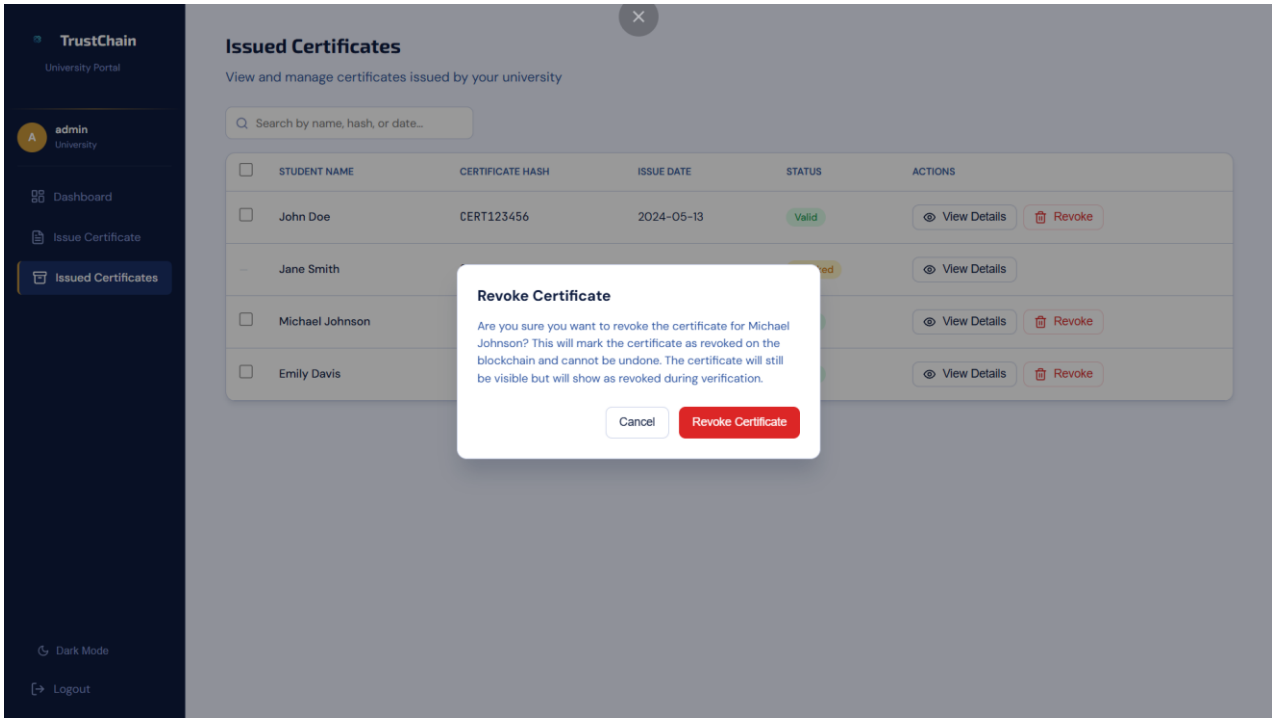



*Figure 4.5.4 Revoke certificate popup*

## 4.6 Technology Used

In Trustchain, we used next generation technology and secure to be up to date, reliable, secure and sustainable.

### 4.6.1 FastAPI

It is a web framework using the python language for building APIs, it is a modern web framework fast and efficient, can work with multiple requests in the same time(async-first).

### 4.6.2 Polygon Amoy testnet

It is a polygon test network and we use it to test the smart contracts in a safe environment, and the coin that used is not a real coin and used for testing purposes.

### 4.6.3 MongoDB + Motor

MongoDB is a NoSQL database, Motor which is a library to make a connection between Python and MongoDB and it is support a asynchronous operations, so it is suitable for work with the FastAPI.

### 4.6.4 Solidity ^0.8.19

A programming language for writing smart contract logic for certificate issuance and verification, used because it is the native language for Ethereum ecosystem mature and well documented and has strong community support and tools, and the smart contract are deploy by using Remix IDE.

#### 4.6.5 Pinata (REST API)

An IPFS service for uploading and store files via pinFileToIPFS, and assign a CID to each file which is used to reach a file that store the certificate in pinata, and fetch via gateway for reaching and uploading the certificate much faster than the public IPFS.

#### 4.6.6 JWT

Each user has a token after login to the system and this token is signed using HS256 and secret key on the system to ensure the data integrity, and for each request to the system this token will be checked, if the signature is invalid the access to the system is denied.

#### 4.6.7 Web3.py

Python library connecting backend with blockchain network that has seamless blockchain integration with the APIs.

#### 4.6.8 React.js

JavaScript library for building interactive user interfaces that hash component-based architecture. It helps in building modern website interface for the portals and verification page.

#### 4.6.9 MetaMask

Wallet and authentication system for user identifier verification.

#### 4.6.10 AES-256-GCM

Symmetric encryption method that is used in our system to encrypt the IPFS CID and PDF bytes.

#### 4.6.11 bcrypt (direct)

Hash method used in our system to hash the password entered by our users (admin or university) in login and compare with the stored hashed password.

#### 4.6.12 SHA-256

Hash method used in our system to produce PDF fingerprint and is converted to bytes32 for use it in the smart contract.

#### 4.6.13 qrcode + pyzbar

for generating and decode the QR-code image.

#### 4.6.14 uvicorn server

this is used to run the FastAPI in our system.

#### 4.6.15 httpx

for Asyn call to Pinata.

#### 4.6.16 pydantic-settings

for reading the setting from .env file.

# CHAPTER 5

# TESTING AND EVALUATION

This chapter is about testing and evaluation for our system, and the purpose of these tests are to ensure that the system is work as suppose as it should work, and these tests are includes:

## 5.1 Smart Contract (SC) Functional Testing.

This table focuses on verifying internal logic of the solidity smart contract uploaded on the Polygon network, ensuring that core functions behave correctly.

| Test Scenario ID | Description | Pre-condition | Expected Result |
|---|---|---|---|
| SC-01 | Issue new certificate | Certificate does not stored before | Certificate stored, Event Emitted, CID created and QR code returned |
| SC-02 | Issue already stored certificate | Certificate already stored | Transaction denied ("Already exists") |
| SC-03 | Verify valid certificate | Certificate stored & valid | Return the status of the certificate + download certificate button + hash value |
| SC-04 | Revoke certificate | Certificate exists & is valid | `status` becomes `Revoked` |
| SC-05 | Verify revoked certificate | Certificate was revoked | Return the status of the certificate + hash value |

*Table 5.1 Smart Contract*

**5.1.1 Description:** these scenarios are implemented to ensure the availability of the core system functionalities, by constructing an anti-tampering contract using solidity smart contract language, including primary functionalities like *certificate_issue, certificate_verify* and *certificate_revoke*, for each party using the system, either the university tries to Issue a new certificate or tried to Issue already stored certificates or the verifier party which might be anyone who want to check this certificate, it will display the result of scanning the QR code which might be *Valid, Revoked and Fake.*

### 5.2 User Acceptance Testing (UAT)

This table focuses on proving the acceptance and reliability in User-Experience (UX) in this system using test scenarios.

| Test Scenario ID | Scenario | Steps | Expected Result |
|---|---|---|---|
| UAT-01 | University issues a certificate | Upload certificate file, student name, degree title, and graduation date -> Click Issue | Success message & certificate QR-code |
| UAT-02 | Verifier checks valid ID | Scan QR code | Green sign: “Certificate is Valid” |
| UAT-03 | Verifier checks fake ID | Scan QR code | Red sign: “Certificate Not Authentic” |
| UAT-04 | Verifier checks revoked ID | Scan QR code | Yellow sign: “Certificate has been Revoked” |

*Table 5.2 User Acceptance Testing*

**5.2.1 Description:** a number of testing scenarios are implemented to ensure the usability to the user parties of our services. First of all, by testing the main function which is to upload certificate, that will lead to pop up a success message after store it in the core system. Secondly, by revealing the test cases of the verifier who wants to check the certificate status, if the certificate is valid a box with green sign will pop up shows a success message. Otherwise, if it is not verified and invalid a box with red sign appears as a sign for not authenticate and invalid, or the last case, the certificate is revoked, a box with yellow sign will pop up shows that this certificate is revoked and fake.

### 5.3 Integration Testing (Backend API)

This section tests the communication between the Backend API (Python) and the Blockchain Service.

| Test ID | Interaction Interface | Test Scenario | Expected Result |
|---|---|---|---|
| INT-01 | Backend API ↔ MongoDB | University Registration: Sending a POST request to /register with university details. | Data is successfully inserted into the application collection in MongoDB, and the API returns a 200 OK status. |
| INT-02 | Backend API ↔ Hashing Lib | Data Integrity: Generating SHA-256 hash for student certificate PDF file within the /issue endpoint. | The system generates a unique, fixed-length alphanumeric string representing the certificate data. |
| INT-03 | Backend API ↔ IPFS Service | Decentralized Storage: Uploading encrypted certificate PDF file with metadata during issuance. | The IPFS service returns a valid Content Identifier (CID), and the backend then encrypts that CID and store the result in (enc_cid). |
| INT-04 | frontend ↔ Smart Contract | frontend: Sending the generated certificate hash with encrypted CID to the issue Certificate function on Polygon. | The Web3 provider confirms the transaction and returns a valid Transaction Hash (TX Hash). |
| INT-05 | Backend API ↔ Smart Contract | Blockchain Read (Query): Calling verify Certificate with a specific certificate hash. | The system retrieves the correct struct data (status, revocation status, issuer address, and timestamp) stored on-chain without gas fees. |
| INT-06 | Backend API ↔ QR Generator | QR Generation: passing the encrypted CID to the QR library. | A readable QR code image is generated and embedded in the API response. |

*Table 5.3 Integration Testing*

#### 5.3.1 Description

These scenarios are implemented to ensure the availability of the core system functionalities, by constructing an anti-tampering contract using solidity smart contract language, including primary functionalities like *certificate_issue*, *certificate_verify* and *certificate_revoke*, for each party using the system, either the university tries to Issue a new certificate or tried to Issue already stored certificates or the verifier party which might be anyone who want to check this certificate, it will display the result of scanning the QR code which might be *Valid, Revoked and Fake.*

# CHAPTER 6

# CONCLUSION

This final chapter provides a comprehensive summary of TrustChain project, highlighting the core achievements and technical findings derived from the development and testing phases. It evaluates the system's performance in addressing academic fraud, identifies the current strengths and limitations of the implementation, and proposes a strategic roadmap for future enhancements.

## 6.1. Conclusion and achievements

Trustchain has successfully demonstrated the feasibility of using the next generation technology such as blockchain and web 3 to establish a secure and decentralized framework for academic certificate verification and replacement of the traditional systems. By using the Polygon network and Solidity smart contracts, the system effectively eliminates the risks that come from centralized databases, such as single point of failure and unauthorized data manipulation.

Key achievements of this work include:

**6.1.1. Immutable records keeping:**

The integration of SHA-256 hashing and blockchain anchoring ensures that if a certificate is issued, it becomes unable to edit or delete.

**6.1.2. Instant global verification:**

The transition from traditional systems that are manual and weeks-long processes to one-click; web-based systems increase the efficiency by far.

**6.1.3. Cost-Effective Infrastructure:**

By using the Polygon network instead of Ethereum, the project achieves high transaction speeds with minimal gas fees, making it a sustainable solution for educational institutes.

## 6.2. Strengths and Weaknesses

### 6.2.1. Strengths

**6.2.1.1. Trust and Transparency:**

The System provides a “gold standard” for authenticity, that restores confidence in the academic ecosystem.

**6.2.1.2. Improved security:**

The use of smart contracts and encryption ensures the CIA Triad of academic records.

**6.2.1.3 User Experience:**

QR code technology implementation makes the verification process easier.

### 6.2.2 Weaknesses

**6.2.2.1Development Environment:**

The project and the current testing are performed in simulated environment (Remix IDE) and test networks; so, the full-scale deployment of it in the mainnet may present scaling and integration challenges.

# REFERENCES


[Reference 1]: https://www.blockcerts.org/about.html
[Reference 2]: https://youtu.be/vioCbTo3C-4?si=trFYWH1vBkqe3kWL
[Reference 3]: https://www.geeksforgeeks.org/software-engineering/blockchain-merkle-trees/
[Reference 4]: https://medium.com/@swastika0015/merkle-proofs-explained-208a72971a50
[Reference 5]: https://github.com/blockchain-certificates/cert-schema/tree/master
[Reference 6]: https://ethereum.org/whitepaper/
[Reference 7]: https://www.geeksforgeeks.org/ethical-hacking/ethereum-gas-and-fees/
[Reference 8]: https://docs.opencerts.io/docs/
[Reference 9]: https://www.openattestation.com/
[Reference 10]: https://www.cloudflare.com/learning/dns/dns-records/dns-txt-record/
[Reference 11]: https://medium.com/coinmonks/how-layer-2-solutions-enhance-blockchain-speed-and-efficiency-8ce1a4f85d06
[Reference 12]: https://www.bcdiploma.com/en
[Reference 13]: https://www.bcdiploma.com/en/blog/an-introduction-to-verifiable-digital-credentials
[Reference 14]: https://www.evidenz.io/img/pdf/BCD-WhitePaper_last.pdf
[Reference 15]: https://www.evidenz.io/bcdt-token.html
[Reference 16]: https://www.geeksforgeeks.org/computer-networks/advanced-encryption-standard-aes/

**APPENDIX A**

# Meeting Minutes

This appendix documents the official meeting minutes between the project team and the advisor. The team held four meetings during the project period: two in person meetings at the advisor's office and two online meetings.

## MEETING 1

**Date & time:** 20/10/2025 2:00pm

**Meeting type:** Online using ZOOM

**Attendees**
Dr. Yaser Jararweh Ahmad Qasim Alqasam Aljorn
**Summary of previous meeting Tasks**
N/A (First meeting)

**What was achieved**
Introduction to the project idea and objectives. Clarification of project scope and expected deliverables. Agreement on project title and general system concept.

**New problem identified**
Lack of clear system architecture at this stage.
Need to refine requirements and technologies to be used.

**Planned Tasks Before Next Meeting**
Prepare initial system requirements. Propose a preliminary system architecture.
Assign roles among team members.

## MEETING 2

**Date & time:** 11/11/2025 2:00pm

**Meeting type:** In person in advisor's office

**Attendees**
Dr. Yaser Jararweh
All project team members

**Summary of previous meeting Tasks**
Define system requirements. Propose system architecture. Assign team responsibilities.

**What was achieved**
Presentation and discussion of system requirements. Approval of the proposed architecture with minor feedback. Confirmation of technologies and tools to be used.

**New problem identified**
Need for further clarification of some requirements. Potential challenges in integrating system components.

**Planned Tasks Before Next Meeting**
Refine requirements based on advisor feedback. Start implementation of main system modules. Prepare UML diagrams.

**MEETING 3**
**Date & time:** 21/12/2025 2:00pm

**Meeting type:** In person in advisor's office

**Attendees**
Dr. Yaser Jararweh Ahmad Qasim Alqasam Aljorn Moath Abed

**Summary of previous meeting Tasks**
Refine system requirements. Begin system implementation. Prepare UML diagrams.
**What was achieved**
Finalization of system requirements. Review of implementation progress. Approval of UML diagrams.
Discussion of report structure.

**New problem identified**
Minor implementation and integration issues. Tight schedule for completing documentation.

**Planned Tasks Before Next Meeting**
Resolve remaining technical issues. Complete system implementation.
Finalize the project report and prepare the presentation.